DevDesk AI 驱动
开发实践报告

一个不懂 C++ 的人如何用纯 AI 做出跨平台桌面软件

日期：2026 年 10 月

编制：DevDesk 项目组

目 次

第1章 引言：这怎么可能？

1.1 一个"不可能"的开局

1.2 真正的悖论

1.3 报告结构：七把钥匙

1.4 阅读前置说明

第2章 技术全景：用了什么、做出了什么

2.1 技术栈：四层结构

2.2 AI 开发环境

2.3 交付物：六个业务模块

2.4 技术全景小结

第3章 人的操作流程：用户从头到尾做什么

3.1 分工前提：代码不可见

3.2 用户的实际操作界面

3.3 用户的操作节奏

3.4 用户不做什么

3.5 纪实案例：UAT6

3.6 小结

第4章 AI 的执行流程：任务的六阶段流水线

4.1 主会话铁律

4.2 六阶段流水线

4.3 调度纪律

4.4 为什么这么重？

第5章 预防机制：用什么解决了什么问题

5.1 防"AI 改了不该改的"

5.2 防"多线并行互相踩"

5.3 防"假绿测试骗人"

5.4 防"静默失败无人知"

5.5 防"忙中出错还看不见"

5.6 防"AI 不走流程"（hooks）

第6章 自动复盘：事故如何沉淀为经验

6.1 经验的三层存储结构

6.2 经验闭环

6.3 cs-retro 辅助机制

6.4 四个生命周期案例

6.5 小结：系统记忆

第7章 AI 行为监控：多层防御的审计体系

7.1 核心设计原则

7.2 四层防御塔

7.3 审计产物标准化

7.4 边界与局限

7.5 纪实案例

第8章 成果与可复制清单：别人怎么照抄

8.1 方法论本质

8.2 可复制清单

8.3 起步指南

8.4 适用边界

8.5 小结

第1章 引言：这怎么可能？

1.1 一个"不可能"的开局

2026年9月，一个不懂C++的人决定做一个跨平台桌面应用。不是"Hello World"（编程入门的第一个程序），不是教程项目——是一个包含笔记、看板、API配置管理（API：应用程序接口——软件之间互相调用的约定格式）、后台AI自动化、仪表盘和系统设置六大业务模块的完整产品，用C++20编写，要同时在macOS和Windows上原生运行。

这个组合看上去像一组互相矛盾的要求，每一条都在把难度推向极端。

"C++20大型项目"本身就是一个高门槛命题。C++是公认的"对开发者要求最高"的主流语言之一：内存由程序员自己管理（什么时候分配、什么时候释放、谁负责清理），编译时间长（编译——把人类写的代码翻译成计算机能执行的指令，C++的翻译过程尤其慢），工具链复杂。"大型"意味着不是单文件脚本，而是分层架构、模块隔离、第三方库集成、双平台CI（持续集成——每次改代码自动编译测试）的工程化项目。"C++20"则是最新一代C++标准，引入了大量新特性，意味着连有经验的C++程序员都需要重新适应。

"用户不熟C++"让局面更极端。代码对用户不可见。用户不写一行代码、不看一行代码、不跑一次编译、不定位任何一个程序缺陷。用户的全部操作界面是：用中文自然语言描述需求、看AI产出的中文文档和截图、拿到安装包后在真机上用"人眼+人手"验收。这意味着所有代码层面的决策——从架构设计到内存管理到跨平台适配——都必须由AI自主完成且不出错。

"六模块全功能交付"则定义了产品的真实复杂度。截至2026-10-21，项目功能清单共110多条，覆盖从富文本编辑器（支持实时预览、全文搜索（输入关键词在全部笔记内容中查找，不只是查标题）、标签系统、目录树管理）到看板拖拽（多列卡片、列内重排、逾期提醒）到API代理（流式转发（数据像流水一样逐段传送，不等全部生成完一次发送）、配置切换、调用日志）到智能体（六种触发方式、工具执行、安全控制（权限闸——决定AI能调用哪些工具、不能调用哪些，像一扇可编程的门禁）、资源预算）的完整业务面。这些不是界面原型——每一个功能都是用户真机点验通过、每天实际使用的代码。

靠一个人？不可能。靠一个不懂C++的人？更不可能。但这就是DevDesk的真实起点。

1.2 真正的悖论：不是"AI能写C++"，而是"没有人能全局掌握它"

AI写C++本身算不上一件新鲜事。Claude Code（一个命令行AI编程助手，能读代码、写代码、跑命令、管理多个并行工作区）可以生成合格的C++代码；配合正确的构建工具链和平台配置，它可以编译、测试、打包出可运行的软件。

但DevDesk在持续高频迭代一个月后——截至2026-10-21共计约 1910+ 次代码提交（截至2026-10-21）——浮现出一个比"AI能不能写代码"深刻得多的问题。

今天的DevDesk，没有任何一个人能全局掌握它的知识总量。不是因为规模大到无法读完——而是因为知识分布在三种完全不同、彼此不互通的载体里。第一类是代码本身（C++源文件、构建脚本、配置），AI生成但用户不读。第二类是设计决策与项目纪律（CLAUDE.md——一个每次AI启动时自动加载的规则文件，以及各期设计文档），记录着每个架构取舍的原因和每条防错规则的来历。第三类是跨项目经验库（约120条从多个兄弟项目中沉淀的可检索经验条目，覆盖交付纪律、反幻觉验证、AI编排、工具链环境等九个领域）。

这三类知识加在一起，总量超出了单一人类的工作记忆限。而项目仍然稳定运转——不是靠人的记性，是靠一套"机制"：规则在每次AI启动时自动加载成为行为边界；独立审计关卡确保改动在到达用户之前至少通过一道不同于实现者的检查；经验从事故中提炼后注入到后续所有会话的上下文中。人不需要记得每一条规则——系统替人记、替人执行。

这就是本报告要回答的核心问题。不是"AI能不能写C++"——答案早就是"能"。而是在"没有人能全局掌握知识"的前提下，"怎么做到"。

1.3 报告结构：七把钥匙

本报告的后续七章，每一章回答一个问题，七章合在一起构成完整的"怎么做到"。

第2章「技术全景」回答"用了什么、做出了什么"——用大白话把C++20、CMake构建系统（CMake——控制"怎样把代码变成可运行的程序"的自动化工具）、EUI-NEO自研GUI框架（一个画界面的跨平台库，相当于网页开发里的React但跑在操作系统原生层）、SQLite数据库、Claude Code多会话协作等技术栈讲清楚（详见第2章），建立读者理解后续各章所必需的基本概念。

第3章「人的操作流程」回答"一个不写代码的用户在开发中做什么"——不是闲着的旁观者，而是在需求定义、关键方向拍板、真机验收三个节点做出不可替代的判断。配完整的流程图，还原用户从"提需求"到"拿到安装包验收"的真实节奏。

第4章「AI的执行流程」回答"AI系统内部怎么把一个需求变成可运行的软件"——六阶段流水线（实现、独立审计、按审计修、测试、合并发布、真机验收），每个阶段的纪律不是设计偏好，而是在具体事故中"撞"出来的。

第5章「预防机制」回答"用什么方法防止AI改错东西"——按"问题到机制到实证"三段式逐条展开。镜像复述（写操作前先复述要做什么等确认）、独立审计关卡（实现者和审查者必须是两个不同的AI会话）、worktree隔离（每个改动在独立的代码副本里进行，互不踩脚）、先红后绿（修复必须先证明测试会失败再修到通过）。每条机制背后都是至少一个真实事故，以及2026-

10-21落地的流程强制层（hooks守卫脚本——把"要求AI走流程"翻译成机器闸门，基线文件被改坏当场拦截）。

第6章「自动复盘」回答"事故怎么变成经验、经验怎么保证下次生效"——经验的三层体系（项目硬纪律、项目软纪律、跨项目经验库）、从事故到条目的完整生命周期、系统如何自动加载经验让AI"动手前已经知道所有已知坑"。

第7章「AI行为监控」回答"一群AI在干活，怎么保证不出乱子"——多层审计防御塔的理念：AI可以犯错，但错误必须在到达用户之前被至少一道独立于实现者的关卡拦住。

第8章「成果与可复制清单」回答"别人怎么照抄这套方法"——提炼出必备机制和起步路径，不止于"我们做到了什么"，更交代"你们怎么做"。

图 1-1：不可能的三角与七轴机制

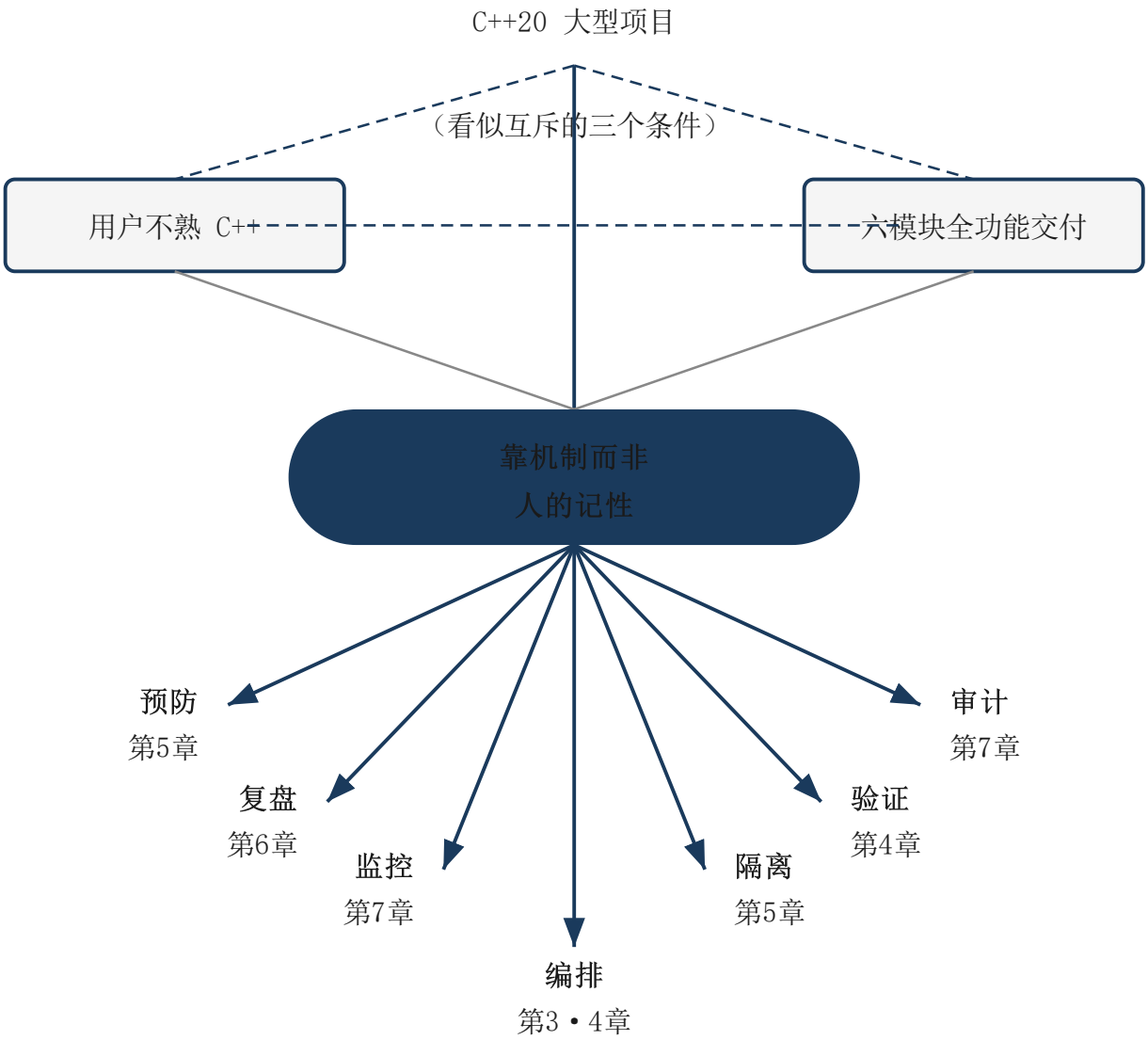


图 1-1　不可能的三角与七轴机制

七章各有侧重，但指向同一个答案：让项目稳定运转的不是某个人的全能，而是一套"机制"——规则在启动时自动加载、改动在被接受前接受独立审查、经验在事故后自动沉淀并在后续所有会话中生效。

1.4 阅读前置说明

本报告记录的全部经验来自真实的开发过程，不是推演，不是最佳实践假设。时间线上需要区分为两层。

第一层，本项目（DevDesk）自2026年9月20日起，经历了一个月的密集迭代开发，截至写作当日（2026-10-21）共计约 1910+ 次提交（截至 2026-10-21）、约1600多条测试（截至2026-10-19全量）、110多条功能点（截至2026-10-21清单口径）。在这一个月中，项目从零走到六模块全功能交付。

第二层，承载这些机制的经验库则根植于更早的实践。自2026年4月起，用户在多个兄弟项目（后端服务、桌面应用、自动化工具）中持续使用AI辅助开发，积累了一批"AI在什么情况下会出什么错、用什么规则能防止它再犯"的经验。这些经验经过去重、提炼、分类后，形成了约120条跨项目共享条目，保存在本报告的"经验库"（一组每次AI启动时自动加载的规则文件）中。

报告中每个机制都会标注其来源——【本项目】表示该机制诞生于DevDesk自身的事故；【方法论沿袭】表示该机制来自兄弟项目的教训，经经验库继承到DevDesk。读者不需要记住哪些是"自己的"哪些是"借来的"——机制好用与否取决于它防了什么事故、效果如何，不取决于它从哪里来。

最后交代一句读者定位。本报告面向两类人：一是用户本人——不熟悉C++、不读代码，需要零黑话、零代码片段的解释；二是团队或外部同行——需要一本可展示的正式技术报告。主体行文兼顾二者：术语首次出现就地用大白话解释；不贴任何代码片段；每个数字都带日期标注；传统报告文风，段落叙述为主。

第2章 技术全景：用了什么、做出了什么

DevDesk 是一个用 C++20 编写的跨平台桌面应用——C++20 是 C++ 语言的最新标准版本（2020 年发布），相当于这门语言的"最新版语法规则"。目标平台是 macOS（Apple Silicon）和 Windows 11。在第1章交代的技术背景之上，本章专注于铺陈构成项目的技术积木与开发环境——四层代码分层、AI 多会话并行的工作方式、以及最终交付的六模块全功能成品——为后面各章讨论"怎么做到的"建立共同的认知基础。

2.1 技术栈：四层结构，每层只管自己的事

DevDesk 的代码组织遵循一种严格的四层分层设计。理解这四层，是理解整个项目技术架构的钥匙。

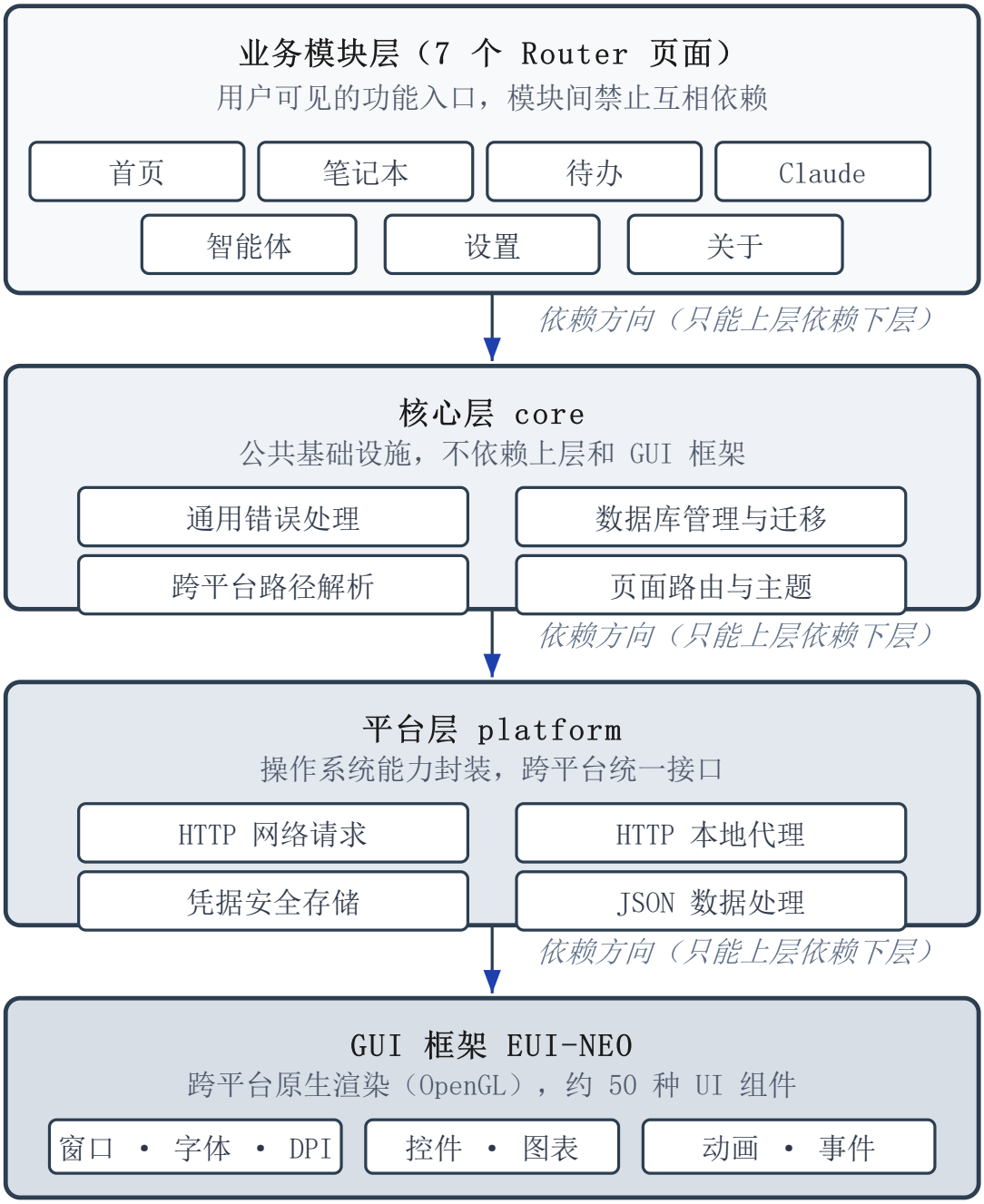


图 2-1 DevDesk 四层技术架构分层框图

最上一层是业务模块层，也就是用户实际能看到、用到的功能入口。目前共有七个页面模块（含设置、关于，其中四个可插拔，三个恒注册）：首页仪表盘、Markdown 笔记本、Todo 待办看板、Claude 配置切换器、智能体任务系统、设置页、关于页。每个模块是一个独立的"房间"——各有自己的数据、自己的界面逻辑、自己的测试用例，彼此之间不能互相调用。这种隔离不是技术洁癖，而是经验教训的产物：当多个功能模块互相依赖时，改一个模块就可能波及另一个模块的界面出错，而这类错误在 AI 并行开发多个模块时尤其难以在提交前发现。七个模块中，首页、设置、关于这三个模块被设计为"恒注册"——用户不能关闭它们；其余四个模块（笔

记、待办、Claude、智能体）可以通过设置页的开关自由启用或禁用，禁用后该模块的界面入口和后台逻辑完全卸载。

向下一层是核心层（core）。这一层提供所有模块共享的基础设施，但它自己没有任何"业务知识"——不知道什么叫做笔记、什么叫做任务。它只提供最通用的能力：统一的错误处理机制（为什么不用 C++ 传统的抛异常？因为 AI 在调用链中容易漏掉异常捕获点，返回一个"成功或失败"的结果值比异常路径更容易被 AI 正确覆盖）、SQLite 数据库的打开与迁移管理、文件路径的跨平台解析、应用主题色的管理、以及页面路由的注册与分发。核心层的设计红线是"不能依赖上层"——它不能调用平台层的网络请求，也不能使用 GUI 框架的任何运行时功能。这条红线保证了核心层在任何时候都可以被单独测试，不受平台差异和界面框架变动的影响。

再下一层是平台层（platform）。这一层的职责是包装操作系统特有的能力，对上提供统一的、跨平台的接口。例如：HTTP 网络请求（基于 libcurl 库，负责所有对外通信——Claude API 调用、网页内容抓取、GitHub 仓库同步都经过它；OpenSSL 负责这些通信的加密，确保数据在网络传输中不被窃听）、本地 HTTP 代理服务（基于 cpp-httplib 库，让 Claude Code 客户端把 API 请求先发到 DevDesk，经流式转发到真实的 AI 服务端点）、操作系统的安全凭据存储（macOS 上走钥匙串 Keychain，Windows 上走 DPAPI——后者是 Windows 系统自带的数据保护接口，用当前登录用户的身份信息做加密密钥，保证 API 密钥不会被明文读取）、以及 JSON 数据的解析与序列化。平台层的物理文件组织也遵循一条严格的纪律：头文件定义平台无关的接口，具体实现按操作系统拆分成独立文件（Windows 的实现文件后缀为 `_win`，macOS 的为 `_macos`），编译时由构建系统按目标平台自动选择。业务代码中不允许出现 `#ifdef Windows` / `#ifdef macOS` 这样的条件分支——这条规则来自一个反复出现的事故模式：AI 在某个平台上写完代码后，往往只在当前平台编译验证，另一平台的代码路径从未被执行过，直到合并后才在 CI（持续集成，指代码提交后自动触发构建与测试的机器人）上暴露错误。

最底层是 GUI 框架 EUI-NEO。这是整个技术栈中比较特殊的一层——它是 DevDesk 项目维护的一个开源跨平台界面库（基于 sudoevolve/EUI-NEO 的定制分支），相当于网页开发里的 React 或 Vue，但运行在操作系统的原生图形层上——通过 OpenGL（操作系统提供的一套标准画图指令，直接操控显卡渲染界面）来绘制所有控件。它提供了应用开发所需的基础界面控件：按钮、输入框、下拉菜单、开关、表格、图表、对话框、侧边栏、虚拟列表等，总计约 50 种可用的组件类型。同时它也处理窗口管理、字体加载、DPI 缩放、鼠标与键盘事件分发等桌面应用最底层的脏活累活。DevDesk 不把 EUI-NEO 当作一个"拿来用就行"的黑盒子——项目中有一条明确的上游贡献政策：凡是遇到 EUI-NEO 自身缺陷造成的界面问题，优先在 EUI-NEO 仓库里修复，再让 DevDesk 受益。这种做法避免了"在应用层打补丁绕过框架 bug"的常见陷阱——补丁会不断堆积，最终让项目绑死在框架的一个特定版本上无法升级。

四层之间的依赖铁律：只能上层依赖下层，不能反过来。业务模块可以调用核心层和平台层的任何功能；核心层不能依赖平台层，也不能使用任何 GUI 框架的运行时行为；平台层可以调用

核心层，但不能调用业务模块。在上面四层之外，还有一个最顶层的入口文件（main.cpp），负责初始化窗口、注册所有模块、启动应用主循环。这个文件是整个程序的"总开关"，但它不包含任何业务逻辑。

为什么需要这么严格的分层？ 在纯人工开发的项目里，分层是一种常见的最佳实践，但通常依赖工程师的自我约束来维持。在 AI 驱动的项目中，这条纪律的紧迫性急剧上升：AI 在修改代码时天然倾向于"哪里能用就往哪里写"，如果没有硬性的分层规则约束，它会在一个本该只处理网络请求的文件里顺手塞进一段界面更新逻辑——当时看起来省了一步，但当这个网络模块需要被另一个界面模块复用时，那段"顺手塞进去"的代码就成了污染源。分层的本质不是增加复杂度，而是用结构代替记性——边界清晰了，AI 在每条边界内部的犯错成本就局限在该层内部，不会蔓延成跨层的连锁错误。

2.2 AI 开发环境：不止一个 AI 在工作

理解 DevDesk 的技术全景，不能只看代码本身——还需要理解 AI 是如何与这套代码交互的。

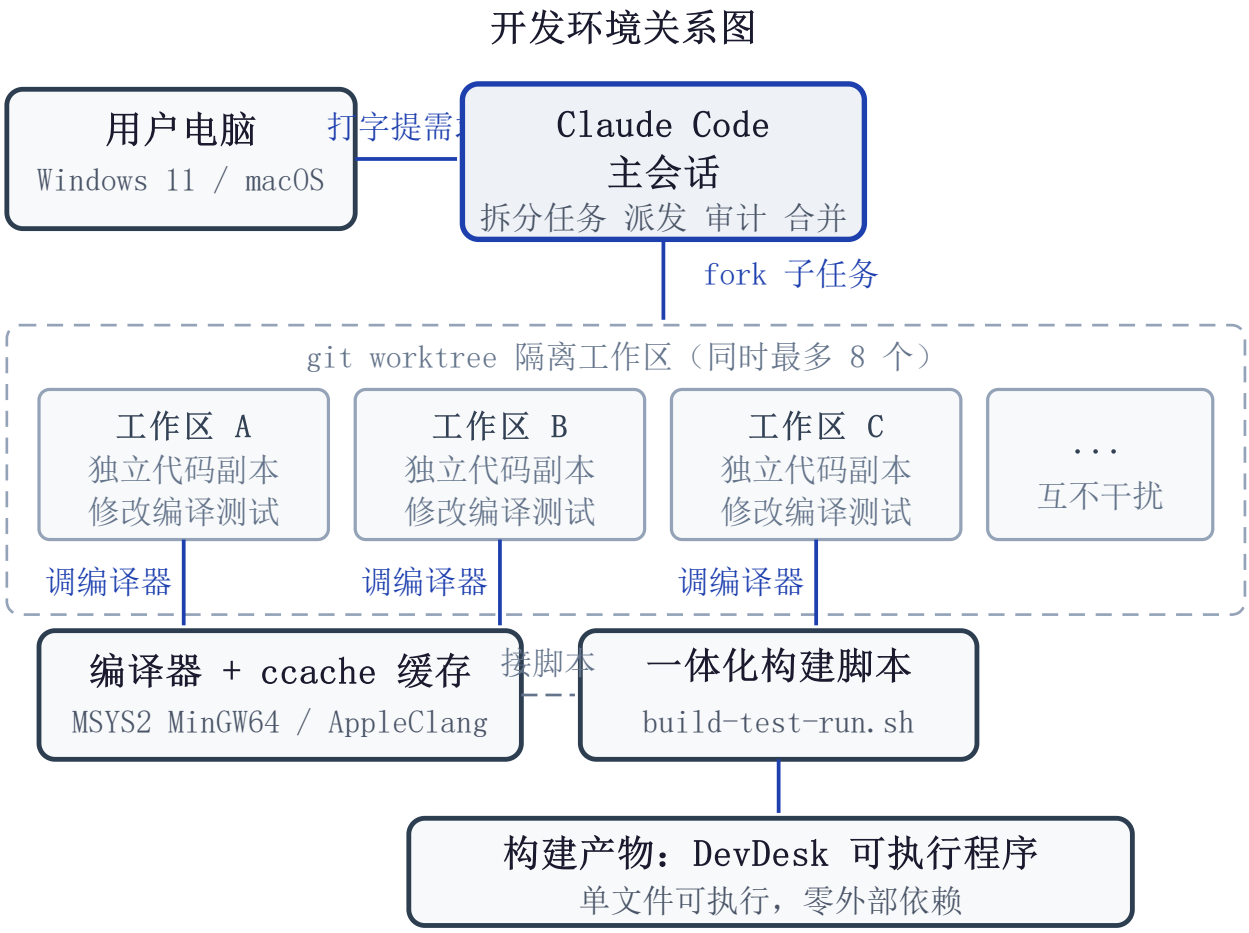


图 2-2 AI 开发环境多会话并行工作示意图

AI 编程助手的角色：DevDesk 使用的 AI 编程工具是 Claude Code——一个命令行界面的 AI 编程助手，能够阅读代码仓库、写代码、运行构建命令、派发子任务给其他 AI 会话。用户通过自然语言向它描述需求（"加一个导入文件的功能""把设置页移到侧边栏"），它在理解意图后自主完成从代码编写到测试验证的全流程。用户本身不写代码、不读代码、不跑编译——这在第 3 章会详细展开。

多条 AI 会话并行工作：实际开发中，同一时刻往往有多条 AI 会话在并线干活——一条在修笔记模块的搜索 bug，一条在给智能体模块加新工具，一条在做全量代码审计。它们共用一个代码仓库，但不能互相干扰。解决这个问题的机制叫 git worktree（工作区隔离）：每当 AI 接到一个需要修改代码的任务，它就在独立的工作区副本中工作——相当于从主代码仓库复制出一份临时副本，在那里面修改、编译、测试、提交，全部完成后才把结果合并回主仓库，随即删除临时副本。这意味着每个 AI 会话都活在自己的"沙盒"里——它看到的文件状态是干净、一致、不受其他 AI 干扰的。这个机制是 DevDesk 能够同时跑 8 个并发子任务而不互相踩脚的基础。

编译与缓存：DevDesk 在 Windows 上使用 MSYS2 MinGW64 编译器（GCC 的 Windows 移植版），在 macOS 上使用 Apple 自带的 AppleClang 编译器。从源码到可运行程序的构建过程由 CMake（一个跨平台的构建管理和配置工具，负责描述"哪些源文件编译成哪些目标、要链接哪些库"）驱动。但 AI 开发有一个独特的问题：每次派一个新任务就要建一个新的工作区副本，如果每个副本都从头完整编译，时间开销会让开发节奏不可接受。解决方案是 ccache（编译缓存）：它会记住每次编译的结果，当同一个文件在另一个工作区里再次被编译且内容没变时，直接复用上次的结果，跳过实际编译步骤。在没有缓存时，两个工作区的同一份源码会被认为"不同"（因为它们的绝对路径不同，而编译命令里包含了路径），导致缓存永远无法命中——解决这个问题的关键技术细节是让缓存把不同工作区的路径"翻译"成同一个逻辑路径后再比对。这个话题的技术细节超出了本报告的讨论范围，但其后果对开发节奏至关重要：有缓存时，一个全新工作区的构建时间从十分钟级降到一分钟左右。

双平台 CI 守卫：每次代码提交到集成分支时，GitHub 的自动化流水线（CI，即持续集成）会在 macOS 和 Windows 两条独立的虚拟机上各跑一次完整的"构建 + 全量测试"。如果任一平台失败，提交会被标记为不可合并。这条机制是 DevDesk 能够同时维护双平台而不出现"这边修好了那边崩了"的基础保障——它不是靠人记着"Windows 上也得验一下"，而是靠机器自动拦。

2.3 交付物：六个业务模块全功能运行

下表汇总了 DevDesk 目前交付的六个核心业务模块（关于页作为信息展示页不纳入业务模块统计，但在整体架构中与设置页同为恒注册页面）。表中功能计数口径为 `docs/strategy/01-业务功能清单.md` 同模块标注为"可用"的功能行数，不含标注为"规划中"的项；智能体模块的工具按注册条目分别计数。每个模块都有独立的界面页面、独立的数据存储、以及一套覆盖其核心场景的自动化测试。

表 2-1：六大模块功能与成熟度一览

模块	一句话功能	当前状态
首页仪表盘	打开应用的第一屏，用统计卡片、柱状图和快捷入口展示当天的工作全貌	10 项功能可用——统计四卡、API 速率、柱状图、快捷跳转、矮窗自适应等
Markdown 笔记本	写作工具，支持编辑、实时渲染、双栏、预览四种视图，带全文搜索和目录管理	20 项功能可用——四态视图切换、全文搜索（SQLite 内置 FTS5 引擎，支持中文子串匹配）、标签系统、目录树折叠记忆、大纲面板、外部修改检测、自动 git 提交等
Todo 待办看板	多列卡片管理，支持拖拽跨列、列内排序、优先级、标签、逾期提醒	15 项功能可用——多列可配置看板、拖拽跨列移动、列内拖拽重排、逾期视图、GitHub Sync 联动等
Claude 配置切换器	管理多套 API 账号配置，启动本地代理实现一键切换 Claude Code 使用的模型服务	14 项功能可用——Profile 增删改查、本地代理启停与配置接管、SSE 流式透传（一种让数据边生成边发送的网络协议，聊天 AI 逐字出字就靠它）、调用日志、共享配置片段、代理自启与端口配置等；另有 1 项（VPN 自动恢复）尚在规划中
智能体任务系统	让 AI 在后台按时间表或事件自动执行任务——相当于一个可编程的自动化机器人	35 项功能可用——六种触发类型（含 cron 定时调度——一种用简洁表达式指定"每周几、几点几分"自动执行的方式）、AI 草稿生成、运行审计、15 款注册工具、双通道失败通知、资源上限强制等
设置与关于	主题色/字号/窗口效果/模块开关等全局配置的集中管理；版本与构建信息展示	设置 15 项 + 关于 2 项功能可用——三档主题模式、四套主题色、窗口效果（含磨砂）、模块开关即时生效、GitHub 仓库同步、代理自启、日志管理等

> 智能体模块的 35 项可用功能含任务管理、六类触发调度、工具执行、运行审计与通知全线——其中 AI 工具集共 15 款（笔记创建/列出/删除、待办创建/移动/完成/列出、Claude Profile 管理、连接测试、日志查询、应用统计、系统通知、网络抓取、热榜聚合、当前时间），覆盖了笔记、待办、Claude 三个业务模块的完整工具面，使智能体能够通过自然语言指令实际操作 DevDesk 自身的所有功能。

2.4 技术全景小结

把上面这些信息收束成一句话：DevDesk 的技术基底是 **C++20 原生桌面应用**，靠四层分层的代码结构维持模块之间的边界清晰；它的开发环境是 **多条 Claude Code AI 会话并行工**

作，靠工作区隔离和编译缓存保证效率；它的交付物是 **六个全功能业务模块加一个信息页**，覆盖了笔记写作、任务管理、AI 配置切换、后台智能体自动化四类核心场景，在 macOS 和 Windows 上均有原生的安装包交付。

这些技术选择背后有一个贯穿始终的设计原则——本文称之为"为 AI 开发而设计"：每一项技术决策都不仅回答了"这个技术能不能做到"，更回答了"这个结构在 AI 不断修改代码的过程中，能不能自己守住自己的边界"。分层结构守住了模块之间的边界，平台抽象守住了操作系统之间的边界，工作区隔离守住了并发任务之间的边界，编译脚本封装守住了构建路径的边界。在全部章节中，本章只负责画地图——把"有什么"铺在桌面上。从下一章开始，报告将逐一展开"怎么做到的"——人怎么操作、AI 怎么执行、机制怎么防错、事故怎么变经验、系统怎么审计、别人怎么照搬。

第3章 人的操作流程：用户从头到尾做什么

3.1 分工前提：代码不可见

DevDesk 的开发有一个先决条件，它定义了整个协作模式的基础：代码对用户不可见。用户不写任何一行代码，也不读任何一行代码。这不是偏好——这是项目启动时就写进项目文档的铁律，CLAUDE.md 第一条核心约束就是"用户不熟悉 C++，纯 AI 开发"。

这意味着用户的工作面与程序员完全不同。用户面前没有集成开发环境（IDE，即程序员写代码、调试、运行的专用软件），没有代码编辑器，没有命令行终端。用户的工作面只有三样东西：

中文文档。 设计方案、进度报告、审计结论——全部用不会被误解的中文写成。不需要读者懂任何编程概念，每份文档的判断标准是"不查任何资料一遍读懂"。这是项目写作的铁律，因为文档的唯一读者就是用户本人——一个不读代码的人。

截图。 AI 在完成改动后自动截取界面状态，保存为图片文件。用户在不启动软件的情况下就能看到改动的视觉结果——按钮位置对不对、颜色顺不顺眼、间距有没有乱。这省掉了用户"装包、启动、点开对应页面"的等待时间。

可运行的软件安装包。 Windows 上是双击就能安装的 exe 文件，macOS 上是一个可以直接拖进"应用程序"文件夹的 app 安装包。用户不需要编译，不需要配置环境，不需要安装任何开发工具——拿到文件，双击，就能用。

所有代码层面的决策——架构怎么设计、函数怎么写、用什么技术方案、bug 怎么修——由 AI 自主完成。用户只在两个节点介入：需求定义和结果验收。这和传统软件项目中"产品经理定义需求、程序员负责实现、质量工程师负责测试"的分工逻辑是一样的，只是"程序员"和"质量工程师"的角色从人类变成了 AI。

这个分工前提贯穿本章全部内容。如果不先把"代码不可见"和"用户只介入两端"交代清楚，后面讲用户的每一步操作就没有上下文。

3.2 用户的实际操作界面

用户与 DevDesk 开发过程的交互，通过三种"界面"进行。这三种界面没有一个是代码界面。

第一种界面：Claude Code 的对话窗口。 这是用户和 AI 系统之间的主要沟通渠道，看起来像一个聊天软件——用户打字，AI 回复。用户在这里做的事情只有三件：(1) 用自然语言描述想要什么，可能是一句话，可能是一段方向性描述；(2) 在 AI 给出选择题或方案时，选定一个方向；(3) 看 AI 回来时的完成报告，了解改了哪些、截了哪些图、装包装在哪里。用户不在这个窗口里写代码，也不在这个窗口里跑命令——它纯粹是一个"对话界面"。

第二种界面：操作系统的文件管理器。 用户通过文件夹窗口查看 AI 交付的产物——截图存在哪个目录、安装包下载到哪个位置、双击哪个文件安装。这个界面上用户的操作就是日常使用电脑的常规动作：打开文件夹、看图、双击运行。

第三种界面：DevDesk 应用本身。 这是最终交付物——用户在自己电脑上安装并实际使用的桌面软件。在这个界面上，用户的身份从"开发项目的管理者"变成了"软件的普通使用者"——点按钮看有没有反应、拖拽卡片看交互对不对、输入文字看排版有没有乱、缩小窗口看内容有没有被截断。用眼睛和手来判断"看起来对不对、用起来顺不顺"。这是代码测试永远无法替代的最后一道质量关卡，原因是自动化测试只能验证"程序行为与预期一致"，验证不了"人看着舒服不舒服"。

三种界面各自的用途不重叠：对话窗口管"方向和决策"，文件管理器管"接收交付物"，应用本身管"最终验收"。用户不需要在三种界面之间做复杂切换或信息同步——每种界面的操作模式都是日常使用电脑就会的。

3.3 用户的操作节奏

整个开发过程中，用户的操作遵循一个固定的节奏，包含五个阶段。大多数需求都会在这个五阶段循环里转上一到多轮才最终完成。这个循环的全貌见本章最后一节的流程图（图3-1）。

阶段 A：提需求，给方向

用户用自然语言发起一切。可能是一句话——"把设置页放到侧边栏"、"这个按钮为什么没反应"、"字体太小了"；可能是一段方向性描述——"智能中枢应该走无人值守、桌面常驻、业务绑定这三个方向，这是贯穿所有开发会话的设计基准"。

不需要写规格书，不需要画原型图，不需要考虑技术可行性。那些判断由 AI 在收到需求后自行完成——它能读取整个代码库的结构和历史，知道哪些能做、哪些有冲突。

这个阶段的产物就是一段自然语言文本。用户说完就完了。AI 会自行判断可行性，并在确实不清楚用户意图时追问一句——追问只限于"真搞不清楚你到底要什么"，因为项目纪律里有一条专门约束 AI：用户已经明确讲过的事实，不准反复问。这条纪律叫"既定事实不再二次追问"，来自跨项目经验库中多个项目共同踩过的坑——AI 习惯性地用"你确定吗"、"那某某是不是也应该"式的追问，在用户看来是浪费时间。

阶段 B：关键节点拍板

（本书统一用"拍板"指代用户在 AI 给出的方案或选项中选定的动作，下文不再重复解释。）

AI 接到需求后，会自行完成调研、分析、方案设计，然后把需要用户选择的节点拿出来——通常以选择题或简要方案文档的形式——请用户拍板。

什么是"需要用户选择的节点"？AI 自行处理的范围包括：用哪种技术方案实现、代码文件怎么组织、变量怎么命名、代码风格细节。只有不可逆的、影响用户体验或产品方向的决策，才会上升到用户层面。例如"按钮用绿色还是灰色"、"窗口最小尺寸定为 1000 还是 1280"（决定小屏幕用户能不能正常使用）、"标题栏是保留系统原生还是自己画"（决定磨砂透明效果能否覆盖整个窗口）。

以 2026-10-20 的 UAT 第三轮为例：AI 给出了多项选择题，用户在同一次对话中逐一拍板了三个方向——标题栏改自绘、最小窗口从 1280 降到 1000、目录树折叠功能三处全修。拍板后 AI 就不再回来反复确认，直接进入执行阶段。

拍板的频率不高。回顾 DevDesk 一个月的开发历程，真正需要用户拍板的节点集中在两类：一是产品方向（智能中枢定位、配色方案选择、模块开关机制）；二是 UAT 中暴露出来的"非代码能判断"的取舍（视觉偏好、交互习惯）。日常功能开发中用户几乎不需要参评。

阶段 C：等待执行

拍板之后到结果出来之前，用户有一段"什么都不用做"的时间。AI 在这段时间里独自完成整个实施流水线——派子任务（把一件大活拆成若干小活分别交给不同的 AI 会话去做）、在隔离环境中写代码、跑测试、截屏、打包——这些是第 4 章要详细展开的内容。

用项目文档里的话说：用户"回来的时候结果已经在桌面"。这句话不是夸张——AI 完成实施后会自动把安装包放到约定的位置，把截图存到指定的目录，把完成报告发到对话窗口。用户不需要守在电脑前盯进度，也不需要在 AI 执行过程中做任何干预。

这个"等待"是 AI 驱动开发与传统人类驱动开发之间最直观的区别。在传统开发中，"等待"是一个人等另一群人把代码写出来；在这里，"等待"是一个人等一套自动化流水线跑完。

阶段 D：真机验收点验

（本书统一用"点验"指代用户在真机上实际点击、操作、用眼睛判断"对不对"的验收行为，下文不再重复解释。）

用户拿到安装包后，在自己的电脑上安装并实际使用。注意"自己的电脑"不是开发机、不是测试机——是用户日常工作的那台真实机器。项目文档中把它叫"真机"，因为只有用户的真实环境才能暴露开发和测试环境中测不出来的问题：屏幕分辨率不同、系统缩放比例不同、字体渲染效果不同、甚至电脑休眠后恢复时的状态处理不同。

在真机上，用户以普通使用者的身份操作——点每个按钮看有没有反应，拖拽卡片看效果对不对，输入文字看排版有没有乱，缩小窗口看内容有没有被截断，切一下系统设置看外观有没有跟着变。

这一步是整个开发流程中最不可替代的环节。原因在于：自动化测试套件（即让计算机自动检查程序行为是否与预期一致的一套脚本）只能验证"程序按代码逻辑在运行"这件事，验证不

了"人看着对不对、用着顺不顺"。测试全绿但用户一上手就发现按钮出框、颜色刺眼、字体太小——这种事在 DevDesk 开发中反复发生。最典型的一例是真机 UAT 三连爆（详见第5章条目七）——全量测试全部通过的同一天，三个独立 bug 全在测试视野之外。

这件事直接催生了项目中的一条硬纪律——"测试绿不等于可用"，以及交付时的一条强制要求——交付前三问（存量数据升级验证过吗？真实外网或网关验证过吗？目标平台真机验证过吗？）。答不上来就必须标明"未验证"，不能沉默过关。

阶段 E：反馈问题

用户在真机点验中发现的问题，用自然语言描述给 AI。典型反馈如"这个弹窗的按钮不在画面内"、"这块区域颜色太米黄了"、"这个提示条每步操作都弹一次太吵了"、"窗口最小尺寸还是太大了"。

用户不提供代码位置，不做根因分析，不指出是哪个文件改坏了。用户只描述"在自己的机器上、做了什么操作、看到了什么异常"。AI 自己负责排查根因、定位代码、修复、重新交付安装包。

反馈后，AI 排查修复，用户拿到新包后回到阶段 D 复验。循环持续到用户说"可以了"为止。这个过程通常在一到两轮内收敛——单轮修复就能过的占多数，两轮以上的集中在视觉和交互类问题（因为"看着舒服不舒服"的标准没有绝对数值，可能需要微调）。

从 UAT 第六轮（UAT6，2026-10-20 前后的一次集中验收）的提交记录可以直观看到这个循环的密度：约 20 多个独立提交（截至 2026-10-21）全是对应用户反馈的单项修复——弹窗按钮出框被固定在窗体底部、卡片内间距从 40 收窄到 28、编辑器字号与标签字号统一联动、窗口透明度从 0.60 调到 0.80、toast 提示的 13 处成功类调用全部删除只保留错误提示。每一个提交对应用户在真机上点到的每一个具体问题，而这些问题在自动化测试中没有一个暴露。

3.4 用户不做什么：人机分工的边界

定义"用户做什么"的同时，同样重要的是定义"用户不做什么"。这份"不做清单"本身就是人机分工的边界线：

不看代码。 代码出了什么问题、改动在哪里、怎么实现的——用户不需要知道。项目文档中明确规定"代码对用户不可见"。

不写代码。 一行都不写。所有代码由 AI 产出。

不跑构建。 用户不碰构建工具，不配置开发环境。构建命令、依赖管理、跨平台适配，全部由自动化脚本和 AI 处理。事实上，项目文档中甚至明确规定"禁止手敲构建命令"——因为手工敲命令会造出多份互相不一致的构建目录，是真实事故发生后的硬纪律。

不定位 bug。 用户只描述症状，不追溯根因。"这个按钮为什么没反应"——种种排查工作由 AI 自行完成。

不写测试。 测试用例由 AI 编写和维护。用户不需要知道"测试覆盖率"是多少。项目纪律明确要求 AI 在给用户的交付报告中"不报测试数字，只报场景清单"——因为"1600 多条测试全绿"对用户没有实际价值，反而会制造虚假的安全感。真机三连爆的教训正是：测试全绿不等于产品可用。

不管理代码版本。 代码的版本管理、分支合并、冲突解决，全部由 AI 处理。用户不需要知道 git 是什么。

这份清单的每一条都不是"用户不想做"——是"用户不需要做"。把传统软件开发中人类程序员负责的全部技术工作从用户的职责范围中剥离出去之后，剩下的才是用户的真实操作面：提需求、关键节点拍板、真机验收点验。这也是本章开头"分工前提"从抽象原则落到具体动作的结果。

3.5 纪实案例：UAT6 中的用户视角

以下两个案例均来自 DevDesk UAT 第六轮（UAT6，2026-10-20 前后的密集真机验收），从用户视角还原"反馈-修复-复验"一个完整循环的真实体验。

案例一：弹窗按钮出框

用户在最小窗口尺寸（1000 像素宽、640 像素高）下，打开待办看板的详情弹窗。弹窗弹出来了，但底部的"取消"和"保存"两个按钮看不到——被挤出了弹窗的可视区域。

用户的反馈只有一句话：描述了自己在什么窗口下、打开了什么功能、看到了什么异常。没有说"坐标错了"或"布局逻辑有问题"——这些用户既不知道也不关心。

AI 接到反馈后自行排查。根因在弹窗内部：弹窗本身有一个自适应机制——它不会超过窗口大小，所以组件层面已经把它限制在安全尺寸内了。但弹窗的内容是按照"请求高度 600 像素"来排版底部按钮的，实际可用高度远小于请求高度 600 像素，于是按钮被挤到画面外。修复方案是把按钮固定在弹窗底部、不随内容高度浮动，同时给弹窗内容区域加上滚动条——内容再多也能上下滚动看到，按钮始终贴在底部不跑。

修复完成后，AI 重新打包交付。用户在新包上复验：同样窗口尺寸、同样操作、按钮正常出现在弹窗底部。通过。

这个案例的典型性在于：按钮出框的根因涉及三重换算——窗口逻辑尺寸、客户区实际可用高度、弹窗内部请求高度——即使程序员肉眼读代码也需要推算几层才能发现，更不要说用户。但在 AI 驱动模式下，用户不需要知道这些换算关系，只需要用一句话指出"按钮不见了"。AI 自己去找到三重换算中哪一层没对齐。

案例二：Toast 提示噪音

Toast 是一种短暂弹出的小提示条，用于告知用户"刚才的操作产生了什么结果"。在 UAT6 之前，DevDesk 的待办看板模块在用户做几乎每一个操作后都会弹一条成功提示——"已保存"、"已上移"、"任务已完成"、"已创建任务"、"已重命名列"、"已删除列"、"已调整顺序"……用户每做一步，右下角就弹一条，频繁出现。

用户在真机点验中反馈的原话大意是：这些提示每步都弹，全是噪音。

AI 接到反馈后做了两件事：第一，全量扫描了待办模块中所有成功类提示，一共 13 处（截至 2026-10-20）；第二，把 13 处全部删除，只保留错误和异常类提示——"保存失败"、"搜索失败"、"移动失败"、"尚有关联卡片不可删除"等真正需要用户知道的提示。与此同时，一些边界情况（比如"已经在最上面了，不能再上移"）原来走的是带绿色对勾的成功样式，容易误导用户以为操作成功了，也一并改成了警告样式。

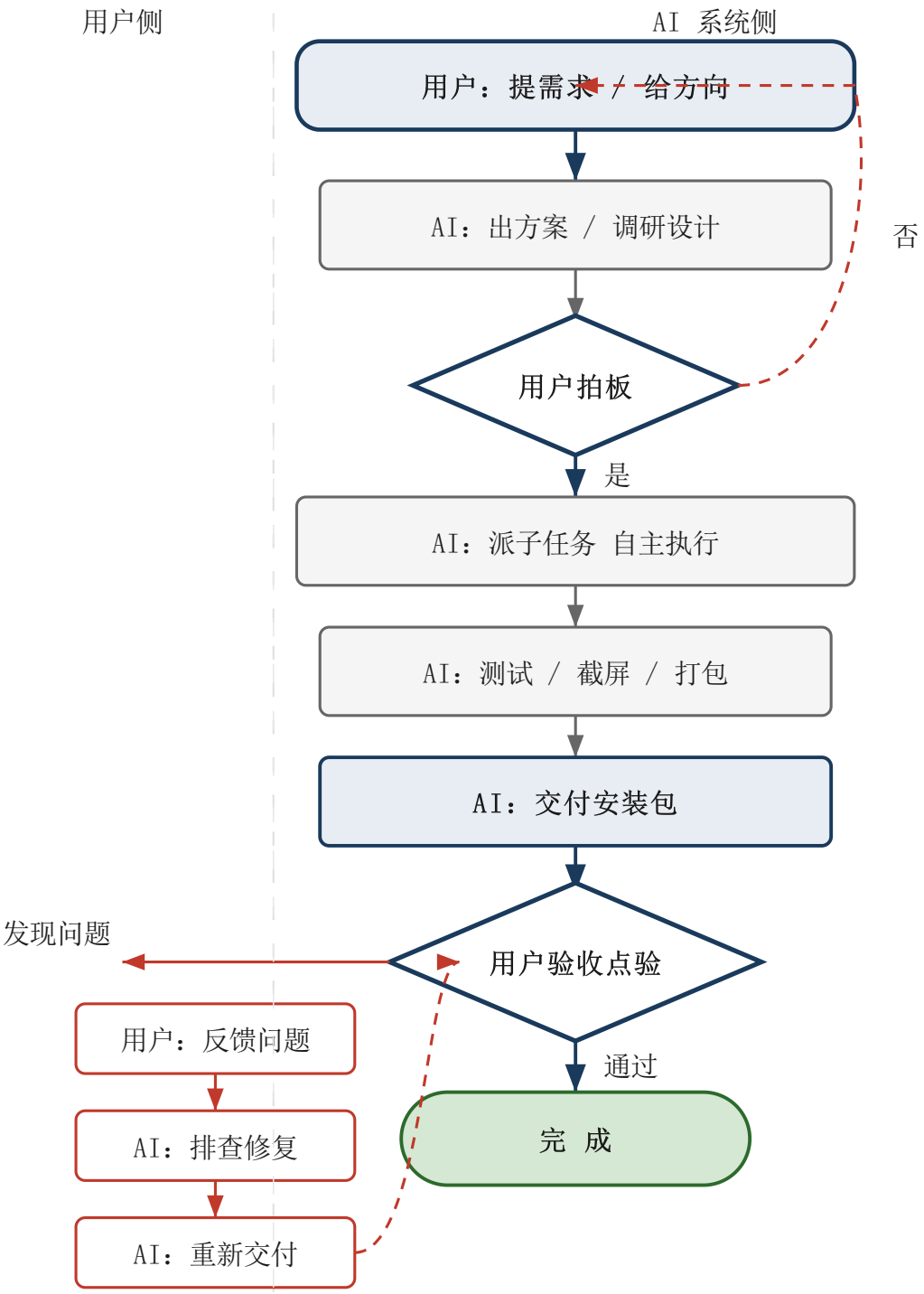
修复后，用户操作待办模块不再被成功提示打断——只有出了问题时才看到提示。这个修改本身很简单（删掉多余的提示调用），但触发它的唯一来源就是用户在真机上的真实感受。自动化测试永远不会告诉你"成功提示太吵了"——从代码逻辑的角度，每一条提示的功能都是正确的，不存在任何 bug，但它对使用体验的损害却是真实的。这正是"代码测试绿不等于产品可用"的又一个活例子。

3.6 小结

本章从五个角度描述了用户在 DevDesk AI 驱动开发中的真实角色：分工前提（代码对用户不可见）、操作界面（三种非代码界面，没有一个涉及代码）、操作节奏（五个阶段的循环，用户介入只在两端）、不做清单（六项技术工作全部从用户职责中剥离）、纪实案例（两个 UAT6 用户反馈的完整循环）。

如果只用一句话概括用户在 DevDesk 开发中做什么：**定义方向、关键节点拍板、最终验收点验。** 代码实现、测试、部署、bug 定位全部由 AI 在机制约束下自主完成——用户既不需要会写代码，也不需要会读代码。

这个模式能运转的前提不是"用户懂技术"，而是"有一套机制在保证 AI 在执行过程中不跑偏"。这套机制的具体内容——AI 怎么做任务分解、怎么过审计、怎么过测试——就是第 4 章要讲的内容。而机制怎么防止事故、每条机制对应的真实教训——就是第 5 章要讲的内容。



菱形 = 用户决策点　　实线 = 主流程　　虚线 = 反馈回路

红色 = 不通过 / 发现问题的路径　　蓝色 = 通过 / 交付的路径

图 3-1　用户与 AI 协作循环流程图

本图采用垂直流程加反馈回路的结构：主流程从上到下依次展开用户与 AI 的交替节点，两个反馈回路（"拍板不通过"回到提需求，"验收发现问题"回到 AI 修复）形成迭代闭环。菱形节点标在用户拍板与用户验收点验两处——它们是整个流程中用户仅有的两次决策点。

第4章 AI 的执行流程：任务的六阶段流水线

> 写作日期：2026-10-21 | 字数：约 5200 字

一个不懂 C++ 的用户对着 AI 对话窗口说一句话。几天后，他的桌面上多了一个可以双击运行的跨平台桌面应用，六个模块功能齐全，双平台构建通过。中间发生了什么？

如果 AI 只是"收到指令 → 写一段代码 → 结束"，DevDesk 不可能在一个月密集迭代中保持稳定。真正让这套系统运转的，是一套被真实事故撞出来的**六阶段流水线**。从用户提需求到安装包交付，一个改动要经过实现、独立审计、按审计修、测试、合并发布、真机验收，六个阶段缺一不可。每个阶段有自己的"谁来做""产出什么""不通过怎么办"，判据不满足就停在原地，不得进入下一阶段。

本章把这套流水线完整拆开，讲清每个阶段的纪律为什么存在——每一条纪律背后，都有至少一个"没这条纪律时出过事"的实证。

4.1 主会话"只编排不实施"的铁律

在讲六阶段之前，必须先讲一条贯穿全局的铁律：**主 AI 会话不写代码**。

"主会话"指的是用户直接对话的那个 AI 窗口——它负责理解用户的需求、把大任务拆成小任务、派给不同的"子任务"（即独立的 AI 工作会话）去执行、验收子任务产出的结果、决定是否合并进主代码库。但主会话自己**不写任何一行代码、不改任何一行配置、不动任何一个文件**。

这不是技术限制——AI 当然能自己改文件。这是一条**硬纪律**，来自 2026-09-27 的教训：当天主会话连续直接修改源码和配置文件，既当运动员又当裁判——改完代码自己判"改对了"，判完就合——结果漏掉了多处本应由第二双眼睛发现的问题。用户当即纠正："既写又审会失去第二方视角。"

这个逻辑可以这样理解：任何人在检查自己刚写完的东西时，大脑会自动填补"我认为我写了什么"，而不是读取"实际写出来的是什么"。程序员自己审自己的代码会漏掉 30% 以上的错误，这在软件工程领域是公认真理。AI 同样如此——实现者对自己的改动有"完成偏见"，会下意识认定目标已达成，而审计需要的是"怀疑偏见"——先假设改动有问题，再逐行找证据推翻这个假设。这两种心态不可能由同一个 AI 会话同时持有。

因此，DevDesk 的开发体系把"实现"和"审查"拆成两个不同的角色，由两个不同的 AI 会话分别承担。主会话的职责是"编排"——拆分任务、派发任务、验收结果、决定合并；子会话的职责是"执行"——写代码、跑测试、做审计。主会话不越界写代码，子会话不越界做审计（审计者

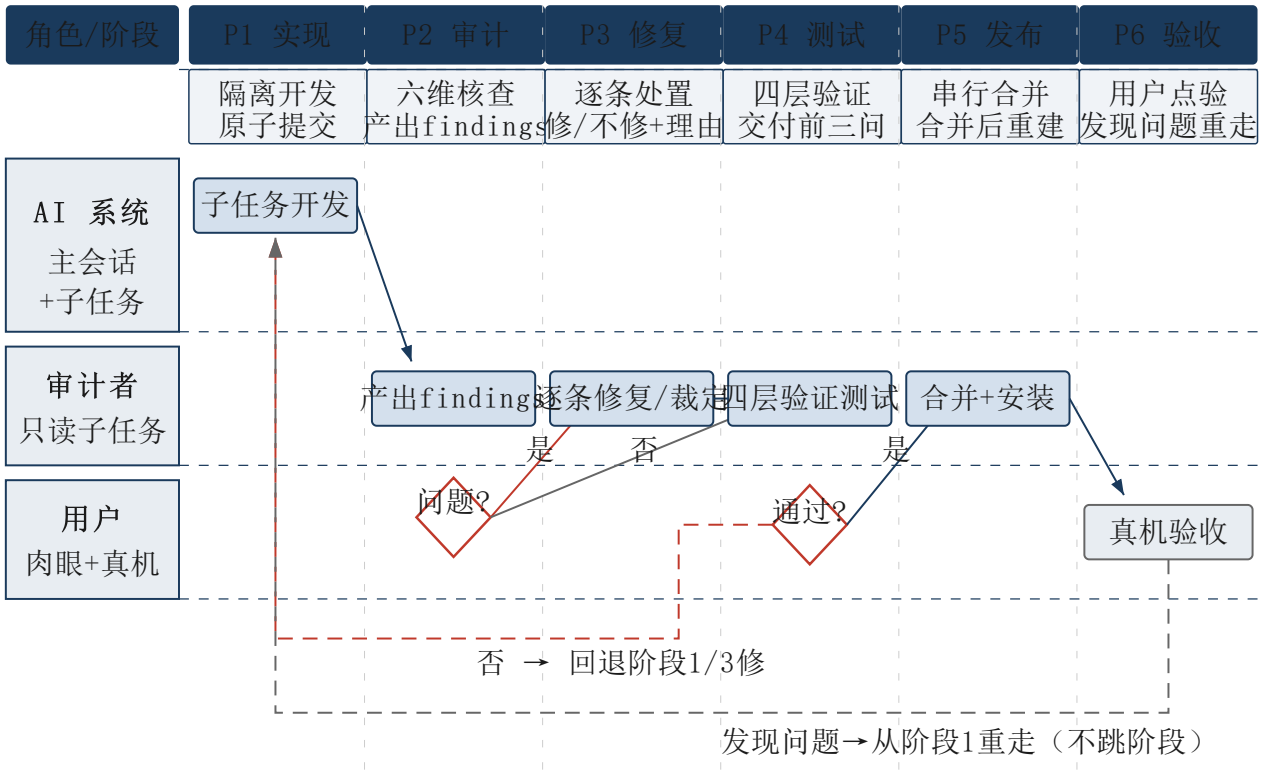
是另一个独立的只读子会话）。这条铁律在后续的每一章都会反复出现——它是整套体系运转的前提。

4.2 六阶段流水线

DevDesk 的每个功能、每个修复、每次改动，都必须在六个阶段中按顺序走完。这六个阶段不是设计出来的——是 2026-09-28 一整天集中暴露的缺陷"撞"出来的。当天接连出现了回归（改 A 坏了 B）、平台不对称（Windows 写的代码在 macOS 上 115 条测试失败）、静默失败（材质从未挂上却一行日志都没有）、假绿测试（断言恒真、永远绿）、文档失实（文档写了不存在的字段名）、交付未合并未安装（修复完了但用户手里还是旧版本）。这些缺陷的共同点是：改动"实现完了"就被当成"做完了"——没有任何中间关卡做验证。

于是，从当天起，每个改动都必须走完六阶段。这六个阶段不是随意排列——顺序本身就是纪律：先审后测（审计确认方向对，测试再验证跑得通），先合后装（合进集成分支通过构建，才覆盖安装），先 AI 后用户（AI 系统的五道关卡全过后，用户才用自己的眼睛做最后验收）。

图4-1：AI六阶段流水线时序图



图注：每个阶段标注产出与门禁判据。通过则箭头指向下一阶段，不通过则停在原地或回退。阶段间流转由主会话调度。详见正文 4.2 节各阶段详述与表 4-1 速查。

图 4-1 AI 六阶段流水线时序图

阶段 1：实现

谁做：派出的子任务（独立 AI 会话），在一个隔离的代码副本中工作。

子任务收到主会话发来的任务描述后，在隔离的工作区（git worktree，可以理解为从主代码库复制出的独立副本，详见第 5 章 5.2）中修改代码、运行初步验证。完成后的交付物不是"口头说做完了"，而是三条硬证据：**分支上的提交记录编号（commit sha）、本次改动了哪些文件的清单、"改了什么、为什么这么改"的文字说明**。三者缺一不可。

"三者缺一"这个要求同样来自事故——早期子任务完成时只口头报告"做完了"，主会话无从判断到底改了哪里、为什么改，结果要么合并了不该合的内容，要么发现缺东西后只能从头排

查。有了 sha + 文件清单 + 改动说明，主会话可以不打开代码就能判断"这个改动是不是我要的"。如果缺了任何一项，主会话不会重新派人重做——而是唤醒同一个子任务，说"你上次停在这里，补齐缺的信息接着做"，避免从头再来的浪费。

子任务的工作区是隔离的——它在自己的代码副本里改，不会影响主代码库，也不会和其他子任务互相踩脚。隔离的必要性将在第 5 章 5.2 详细展开，这里只强调一条：隔离是前提，不隔离就谈不上并行。

阶段 2：独立审计（强制关卡）

谁做：另一个独立的只读子任务——**审计者不得是实现者本人**，也不得参与本轮改动。

这是整个流水线中最关键的一道闸门。上一个阶段的实现者把代码写完了，但这个代码对不对、有没有暗病、会不会影响其他地方，实现者自己看不出来——不是因为能力不够，是因为"完成偏见"（上文 4.1 节已解释）。

审计者是只读的（read-only），只能读代码、读文档、读测试结果，唯一允许的"写"动作是把审计结论记录到审计文件里。它不能改代码——改代码是阶段 3 的事。

审计按六个维度逐条核查：

维度一：正确性。改动是否真的解决了用户原本提出的问题？这里的关键是"对照用户的原始需求核对，而不是对照实现者自己对需求的复述"。实现者在转述需求时可能已经无意中偏了方向，审计者必须回到用户最初说的那句话去找标准答案。

维度二：回归面。改动触及的所有调用点和依赖点是否全部同步更新？实证来源：某次把"最近使用"列表从一个容器搬到另一个容器时，漏搬了锚点标记，导致用户看到的页面 60% 是空白；另一次在代码中间插入一个新函数时，顺手删掉了两段看起来无关但实际被别处引用的代码。审计者必须从改动的每一处向外辐射检查——这个函数被哪些地方调用？这个变量被哪些地方读取？改了这里，那里要不要同步改？

维度三：平台对称性。DevDesk 支持 Windows 和 macOS 双平台。某个改动可能在 Windows 上验证通过，但 macOS 侧的对应代码路径可能完全不同。实证来源：项目早期在 Windows 上开发了一段时间，期间写的 macOS 适配代码从未编译过——等回到 macOS 后首次构建，115 条测试中有 117 条失败（只有 2 条通过）。审计者必须检查：这个改动是否在两个平台上都成立？如果只在单平台验证过，标记出来。

维度四：静默失败面。最危险的一类缺陷——不崩溃、不报错，但也不做正确的事。典型表现：异常被吞掉（catch 了但不记录任何日志，返回一个默认值假装正常）、数值被兜底钳位（一个"高度不足"的问题被 clamp 成"少几行但不会报错"）。实证来源：某次代码中 `catch { return false; }` 让一个渲染材质从未成功加载，但程序照常运行，界面上看不出异常，一行日志

都没有——如果不是后来用户肉眼发现"这里看起来不对"，这个缺陷永远不会被约 1600+ 条测试中的任何一条发现。

维度五：测试有效性。新增的测试断言是不是"恒真的"——不管代码怎么改它都通过？验证方法是"变异验证"：把改动的代码回退到改动前的状态，看测试会不会变红。如果不红，说明这个断言不依赖被改动的代码——它是一个"假绿"的装饰品，不是真正的守卫。实证来源：自定义测试框架中用 `print` 代替 `raise AssertionError` 来报告失败——print 不改变退出码，运行结果显示 "7/7 ALL PASSED"，实际上 7 条都失败了，只是因为 print 不被计为失败。

维度六：文档口径。文档里有没有和新行为矛盾的陈述？实证来源：某份文档写"0 未提交改动"，而实际工作区有 3 个未跟踪（untracked）文件；另有文档写了一个代码中不存在的字段名，后来按文档去排查问题的人被带进了死胡同。

审计产出是一份结构化的"发现清单"（findings）。每条 finding 必须包含精确的文件和行号（`file:line`）、问题是什么、影响了谁、建议怎么修。六个维度必须逐条给出结论——即使"未发现问题"也要写"已核"，不能空着。空着就是没审。此外，如果审计者和实现者是同一个 AI 会话，则审计无效——不管六个维度答得多完整、不管 findings 写得多么细致，因为实现者不可能对自己的代码持有"怀疑偏见"。

阶段 3：按审计修

谁做：派子任务在功能分支上修。

拿到阶段 2 的 findings 清单后，子任务逐条处理。两条路：**修复**，并给出新的提交记录编号；或**明确判定"不修"**，并写明理由——"不修"不是消极回避，是有意识的设计决策（例如"这个 finding 所描述的行为是用户明确要求的设计意图、不是 bug"）。

通过判据很简单：清单上每一条都有处置结论，没有"待定"或空白项。且所有相关 finding 的修复结果由审计方或另一个独立子任务复查通过——不区分"关键"与"非关键"，每一条处置都要被另一双眼睛看过。复查未过的，继续修，不得进入阶段 4 的测试。

阶段 4：测试（审计通过后才准跑）

谁做：在功能分支的工作区内执行。

注意顺序——测试在审计之后，不是之前。这是有意设计：如果先跑测试，测试全绿、实现者宣布"做完了"，审计再做发现——这时修完还得再跑一轮测试，结果发现修的时候又碰坏了别的东西——整个流程变成反复兜圈。正确的顺序是"先审对错、再验证对"——审计确认代码逻辑方向没问题之后，测试再验证"在真机器上跑得通"。

阶段 4 不是只"跑一下单测看看绿不绿"。它有四层验证：

第一层，定向单测：跑和本次改动直接相关的测试用例，用测试框架的子串过滤功能只跑名字含特定关键词的（如修改了笔记模块就跑 `markdown`）。只有改动涉及共享底层（核心库 core、平台层 platform 或公共头文件）时才跑全量。理由是全量测试每轮约两分钟，在频繁迭代中累积时间不可接受——用户明确说过"进度太慢了"。

第二层，真窗口探针（probe）：单测只能验证"代码自洽"——输入什么返回什么。但 DevDesk 是一个 GUI 桌面应用，大量行为只存在于"真窗口渲染"之后——按钮有没有画到正确的坐标？弹窗有没有被其他窗口挡住？拖拽卡片时位置有没有正确更新？这些是单测永远覆盖不到的。探针就是为这个场景设计的：它启动一个真实窗口（含 OpenGL 渲染后端），注入真实的鼠标点击、键盘输入、拖拽事件，然后读取像素坐标与几何数值做断言。凡改动涉及可见 UI 的，必须有探针覆盖——光有单测不算。

第三层，真网 live 测试：涉及调用外部 AI 模型网关的功能（如智能体模块），mock（模拟假端点）只能验证代码的理想路径——它假设网关永远正常返回、模型名永远被接受、token 计费永远合理。但真实网关有自己的方言、自己的超时行为、自己对模型名的甄别规则。实证来源：某次改用网关 mock 全绿的智能体草稿生成功能，在连接真实网关（公司内部模型网关）后当场爆炸——网关拒绝接受模型名中带的 `[1M]` 标记，而 mock 端点完全不知道有这个规则。因此智能体类测试实行两段制：先用 mock 验证理想路径和分支逻辑，再至少跑一次真实模型调用验证"网关真吃这套"。

第四层，交付前三问：这是 2026-10-19 真机用户验收测试（UAT）三连爆之后定下的铁律。当天全量约 1600+ 条测试（截至 2026-10-19 全量口径）全部绿，任何人看到这个结果都会认为"没问题了"——但用户在真机上连爆三个 bug——全在测试覆盖的缝里——这三类事故的完整实证见第5章条目七。因此，从这天起，任何安装包发出或宣布完成之前，必须回答三个问题：① 存量数据（老版本遗留的数据库文件）的升级路径验证过了吗？② 真实外网 / 网关验证过了吗？③ 目标平台真机验证过了吗？答不上的写进"未验证"栏，不许沉默——把不知道的东西坦率说出来，比假装知道更安全。

阶段 4 的不过处理很严格：任何一项测试红 → 回到阶段 1 或阶段 3 修复，修完重跑；发现恒真断言 → 不是删掉断言了事，而是重写一个真正能红的断言。

此外，凡改动涉及可见 UI 的，实现者还必须在交付前自己启动应用、打开对应页面、截图确认"看起来对"——数值断言能证明坐标对，证明不了"看起来对"。这一步不是阶段 6 的用户验收（那时用户还没见到产品），而是实现者的交付前自查——防止把"肉眼一看就错"的版本送到用户手里。历史教训：标题栏无底色、详情面板 60% 空白、卡片缺边距，全都是测试全绿但视觉崩坏的案例，代价是用户拿到手后发现、再走一轮返工。

阶段 5：串行合并与发布

谁做：主会话执行合并。

功能分支经过审计和测试后，合并进集成分支（`dev` 分支）。这一步有两条硬纪律：

第一条：串行合并。 一次只合一条功能分支——多线同时合并会撞 git 的并发锁（`index.lock`——git 用来防止两个人同时操作同一份数据时互相覆盖的临时标记文件，同一时刻只允许一个人修改），导致合并失败。主会话在合并前检查：有没有其他线正在合？有就排队等，不抢。

第二条：合并后重新构建。 这不是重复工作，而是血的教训。2026-09-28 的一次事故（提交记录号 `f274f74`）：审计和测试都在功能分支的 worktree 上通过了，合并进集成分支后没有重新构建就宣布完成——结果集成分支当场无法通过构建系统的配置阶段（cmake configure 失败——configure 是构建工具 cmake 的第一步：检查编译器和依赖库是否就绪、把"哪些文件要编译、按什么顺序"生成一份清单，这步不通过后面编译也无从谈起），直到补救提交（`1ad2ac9`）才修复。根因是：功能分支上的改动和集成分支上其他人同时合入的改动产生了冲突——分别测试都通过，合在一起就炸。同理，"分支绿不等于合并绿"——一个功能在独立分支上跑通了，不代表它和其他分支的代码组合起来也跑得通。方法是只跑一次构建来验证整个集成分支的"健康状态"——看看所有代码放到一起后能不能构建成功。持续时间不会太长，但这短短一步的验证避免的是"整个环境崩了只能回退"的大代价。

合并后的产物通过构建后，覆盖安装到用户的应用程序目录（macOS 上是 `/Applications/DevDesk.app`）。注意一个关键语义：**没合并 = 没交付**。过去发生过"实现了、测试也绿了，但忘了合进集成分支"的事故——用户在旧版本上验收，得出"还是没效果"的结论，而实际上新代码根本没装到他的机器上。另一类是"合并了但装的是旧包"——修复已在集成分支、甚至已推送到远程仓库，但用户桌面上装的还是修复前的版本。对用户来说，没更新到他手里的修复"不存在"（2026-10-19 增补此条纪律）。

因此合并后的"覆盖安装到用户程序目录"不是一个可选的收尾动作，它是"交付"这个概念的组成部分。

阶段 6：真机验收

谁做：用户在真机上点验。

前面五个阶段全部由 AI 系统完成，阶段 6 是唯一需要用户直接参与的一步。测试能验证"数值对不对""几何坐标对不对"——但验证不了"看起来对不对"。

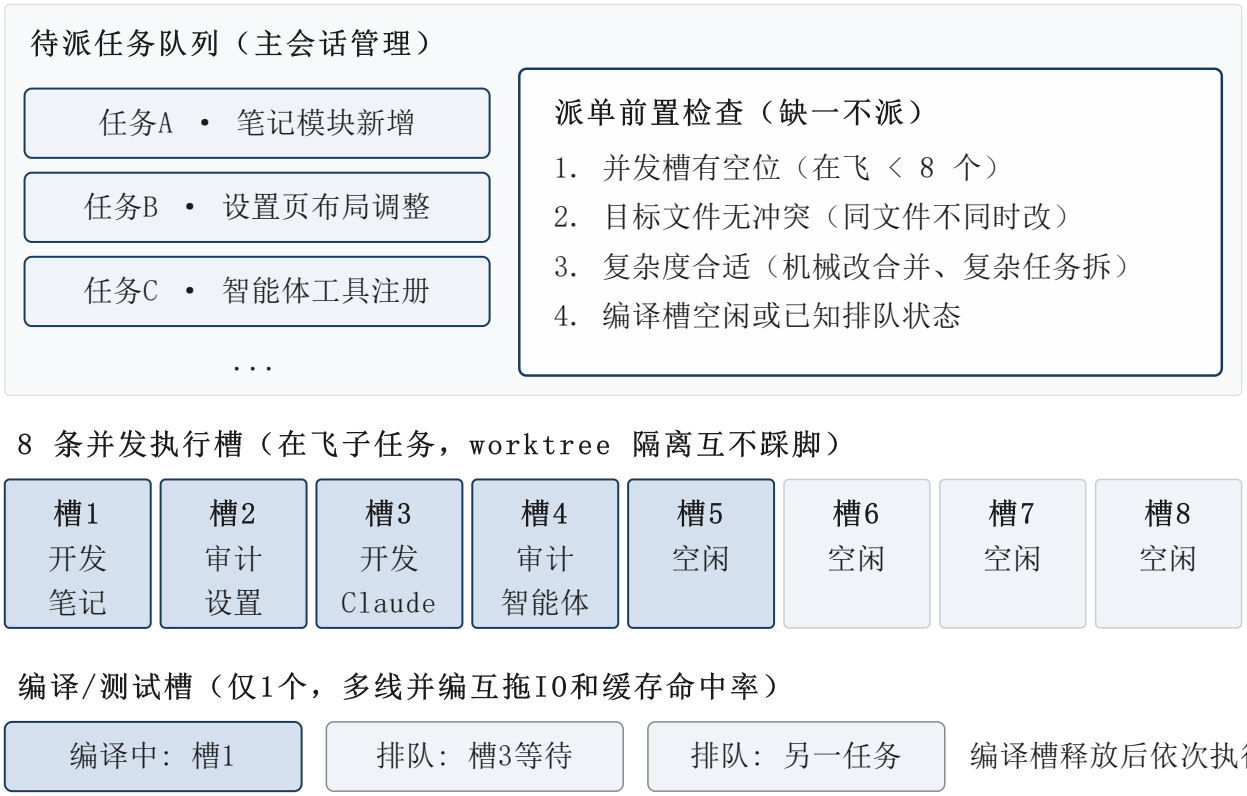
DevDesk 的历史反复证明了这一点：标题栏无底色、详情面板 60% 区域空白、卡片缺少边距、侧边栏边距改错了层级——这些问题全都有对应的测试、测试全都绿，但全都是由用户肉眼先发现、事后才补验证的。阶段 4 的数值断言能证明"按钮在坐标 (100,200)"，但证明不了"按钮有没有出框"——出框的判断需要人眼看"这个按钮的右边缘是不是超出了父容器的右边缘"，这不是某个坐标值能回答的问题，它需要人脑的视觉整体感知。

用户在真机上做的不是"对照验收清单逐条打勾"——他像一个普通用户那样使用软件：点按钮、拖拽卡片、搜索文档、切模块、看页面——用自己的眼睛和手判断"顺不顺手、好不好看、有没有不对劲的地方"。真机验收是代码测试之外的最后一道防线，也是 AI 无法替代的唯一一道防线——因为 AI 没有"看起来舒不舒服"的感知能力。

用户发现问题后，用自然语言描述："这个按钮为什么没反应""这块区域看起来是空白"。他不会定位到代码——这是 AI 的事。AI 收到反馈后走完整流水线：排查 → 修复 → 审计 → 测试 → 合并 → 再交付 → 用户复验。不论问题看起来多小，不走捷径——"小"问题跳过审计和测试，正是历史上最多次"修复一个 bug 引入另一个 bug"的根因。

4.3 调度纪律：流水线之外的系统约束

图4-2：子任务并发调度模型



派一个子任务前检查并发槽是否满（≤ 8）、文件冲突、编译槽状态。8槽可并行读代码和分析，

图 4-2 子任务并发调度模型

六阶段管的是"一个改动的生命周期"，但真实开发场景不会只有一个改动在线——高峰期可能有 8 个功能同时在往前推进。多条流水线并行时，需要在流水线框架之上叠加一套调度纪律，防止"互踩"。

并发上限与编译槽。同时进行的子任务最多 8 个（2026-10-21 由用户从 4 个上调到 8 个），但其中**同时只允许 1 个执行编译或测试**。这不是为了"省计算资源"——12 核机器跑几个编译绰绰有余——而是因为多条线同时编译会互相拖慢 IO（磁盘读写）和编译缓存（ccache，即把编译结果存起来下次复用的工具）的热命中率。实测结果：两条 worktree 同时做增量编译，各自耗时比单独跑长一倍以上，因为磁盘 I/O 带宽被平分，编译缓存的高频读写互相挤占。所以"编译槽只有 1 个"不是限制了效率，是保护了效率——排队等编译比一拥而上更快到终点。

同文件互斥。同一时刻，一个文件只能被一个子任务修改。主会话在派单前，必须检查"当前正在飞的所有子任务都在改哪些文件"——如果新任务要改的文件已经被某个在飞任务占用，新任务就排队等。实证来源：两个子任务（代号 R0 和 H2）同时修改 `settings_module.cpp`，各自在隔离的代码副本中写得都没错，但在合并时其中一个改动被另一个完全覆盖——只能中止其中一条线，重做。

worktree 用完即清。功能分支合并后，立刻删除它的隔离工作目录。不删会怎样？实证来源：一次清理时发现了 12 个残留的隔离工作目录——每个都在磁盘上占空间，每个都可能被误当成"这个 feature 还在进行中"。规则是：合并即清，一刻不留。

构建超时上限。任何命令的超时时间默认最多 3 分钟。长构建（全新编译、全量重建）放后台跑，不靠前台长时间等待。这条例的出发点不是"不要等"，而是"长超时掩盖卡死"——如果一个命令本应该 30 秒跑完但卡了 10 分钟还没结束，把它放在前台干等，等发现是卡死时已经浪费了 10 分钟。如果长超时本身是正常的（如全新 worktree 的首次配置与编译确实需要几分钟），就把它放到后台跑，但必须在报告中说明"为什么这个慢是正常的、不是卡死"。

禁止长时间睡眠等待。单次 `sleep`（让程序原地等待）不超过 60 秒。如果某个步骤需要"等前一步完成"，不用固定长度的睡眠——用轮询（每隔一段时间检查一次条件是否满足）或事件信号（前一步完成时发通知）代替。因为睡眠是"无信息量的等待"——睡了 120 秒，既不能证明前一步成功了、也不能证明它还在跑。而轮询能看见进度、事件能看见终点，信息密度完全不同。

4.4 为什么这么重？

读完这六个阶段和附加的调度纪律，一个自然的疑问是：做一个改动要走这么多步，会不会太"重"了？

回答这个问题，需要回到这套流水线的起源。它不是某个架构师坐在白板前画出来的"理想流程"。它是 2026-09-28 一天之内被五类缺陷轮番打脸之后，连夜定下的"止血方案"——回归、平台不对称、静默失败、假绿测试、交付未合并未安装，每一条都有当天的实证对应。后来的每一次增补（交付前三问、追加消息对账、编译槽唯一）同样是因为"又出事了"才加了一条规则。

换句话说，如果有一天 DevDesk 的开发者觉得这条流水线"太重了想简化"，他简化掉的不是"流程"，是"过去踩过这个坑之后竖起来的护栏"。

整套流水线的设计目的可以用一句话概括：**让一个改动的质量不依赖"实现者有没有想周全"，而依赖"体系有没有拦住实现者想不到的地方"**。实现者可以犯错——人都会犯错，AI 也会犯错。但同一个错误不能同时骗过独立审计者的六维度核查、骗过真窗口探针的像素断言、骗过真网关的 live 测试、骗过合并后的集成分支重建、骗过真机验收环节的用户肉眼——五道独立关卡、五种不同视角、五次独立判断，同时漏掉的概率远低于其中任何单独一道。

本章末尾附一张速查表（表 4-1），方便读者快速检索每个阶段的要素。每一条纪律背后对应的事故编号，指引到第 5 章——在那里，你会看到每个机制被"撞"出来的完整故事。

表 4-1：六阶段流水线速查表

阶段	谁做	产出	通过判据	不过怎么办	对应事故
1. 实现	子任务	commit sha + 文件清单 + 改动说明	三者齐备	唤醒原 agent 补齐	见第5章表5-1（事故"交付物与测试环境隔离"等）
2. 独立审计	独立只读子任务（非实现者）	结构化 findings 文件（六维度逐条结论 + file:line）	六维度全答、每条带 file:line、审计者非实现者	缺 findings → 补；有 finding → 进阶段 3	见第5章表5-1（事故"平台对称性审计"等）
3. 按审计修	子任务	findings 处置完毕清单	每条有结论（已修 / 不修+理由）、复查通过	复查未过 → 继续修	见第5章表5-1
4. 测试	子任务	定向单测全绿 + probe 全绿 + live 通过 + 交付前三问回答	四层验证全过、恒真断言零条	任一红 → 回阶段 1/3；恒真 → 重写	见第5章表5-1
5. 串行合并	主会话	merge commit sha + 合并后构建通过 + 安装到应用程序目录	合并后构建绿、安装到位	构建/测试红 → 回退或修	见第5章表5-1
6. 真机验收	用户	验收通过 / 新问题描述	用户确认通过	新问题 → 按新 bug 重走六阶段	见第5章表5-1

> 注："对应事故"列目前统一写为"见第 5 章表 5-1"占位。事故编号将在合稿阶段（所有章节成文后）统一分配——届时本表各区格填入事故编号（如"Accident-5-3"），第 5 章表 5-1 按编号提供事故摘要与实证来源。编号对齐由合稿编辑统一执行，本章写手不自行编号。流水线本身在 DevDesk 仓库的文档中均有完整记录（`CLAUDE.md`「功能开发生命周期（六阶段）」段与「交付流水线（四律）」段，首次落地日期 2026-09-28），后续各节细则是逐次事故后增补——"交付前三问"增补于 2026-10-19，"追加消息对账"增补于 2026-10-21 hooks 上线同日。

第5章 预防机制：用什么解决了什么问题

实证来源口径（诚实声明）：本章机制的实证分两类——标【本项目】的来自 DevDesk 自身事故（git 与项目文档可查）；标【方法论沿袭】的来自 rs-host、winui3、monitor 等兄弟项目的事故，经跨项目经验库沉淀后被 DevDesk 全额继承为纪律。两类机制在 DevDesk 中同样生效，区别仅在事故的第一现场是不是这个仓库本身。

第4章讲的六阶段流水线是一套"怎么做事"的流程——而本章要回答的是"为什么每一步都非有不可"。每一条纪律背后都站着至少一个真实事故。

DevDesk 的开发过程并非一帆风顺。一个月的密集迭代中，几乎每条纪律后面都站着至少一个真实事故——AI 把不该改的改了、多个 AI 会话同时改同一个文件、测试全绿但用户真机一跑就炸、代码崩了但日志里连一行错误都没有。本章共梳理了 20 条预防机制，选出其中 12 条最有代表性的按"问题是什么 → 怎么防的 → 出了什么事"三段式完整展开，其余条目汇总到章末的表 5-1。建议读者选择自己关心的领域精读对应小节，表5-1提供了全部20条机制的一览索引与事故来源。

5.1 防"AI 改了不该改的"（范围边界机制）

AI 编程助手有一个根深蒂固的行为模式：它会把"看起来顺手可以做"的事情当成"用户的意思"。用户说"把 A 改成 B"，AI 看到旁边还有个 C 长得和 A 很像，就顺手也改了——因为它觉得这样更完整、更统一。这种"自扩展范围"是 AI 驱动开发中最隐蔽也最危险的问题：它不是做错事，是做多了事，而多做的那部分既没有需求背书、也没有测试覆盖，回归面完全不可控。

条目一：镜像复述——写操作前先亮底牌【方法论沿袭】

机制的设计很简单：每次 AI 要动代码之前，先用自己的话复述一遍"我准备改哪里、从什么改成什么、为什么这么改"，等用户确认了再动手。

这个机制的起源不在 DevDesk，而在兄弟项目 rs-host 的开发经历中。AI 多次因为"既然改了 A 顺便 B 也…"而把单点操作扩展为批量修改——比如用户让它在某个 Service 类里加一个字段的输出，它顺手把同目录下三个其它 Service 类里的同名字段也一起改了，理由是"它们长得一样、应该统一"。这些"顺手"改动没有一个经过测试，事后排查回归面的时间比做需求本身还长。用户最终定下铁律：写操作前必须镜像复述，而且分类处理——skill 调用和只读命令不需要复述，但任何改代码、改数据库、跑不可逆命令的动作都必须在动手前把"要干嘛"说清楚。用户没回应就停住，不假设同意。

镜像复述防的不是 AI 做错事，防的是它做多余的事——迫使 AI 在动手之前把意图压缩成一句话，用户当场就能发现偏差。问题消灭在还没发生的阶段，比事后审计省十倍成本。

条目二：只修需求交付物缺陷——审计发现的三分类法则【方法论沿袭】

独立审计（第 4 章的阶段 2）会产出大量 findings——代码里这里不好、那里有隐患。但审计报告里的发现不等于全部都要修。如果审计员报多少修多少，开发范围就会失控：审计员不是需求方，他的"建议"替代不了用户的授权。

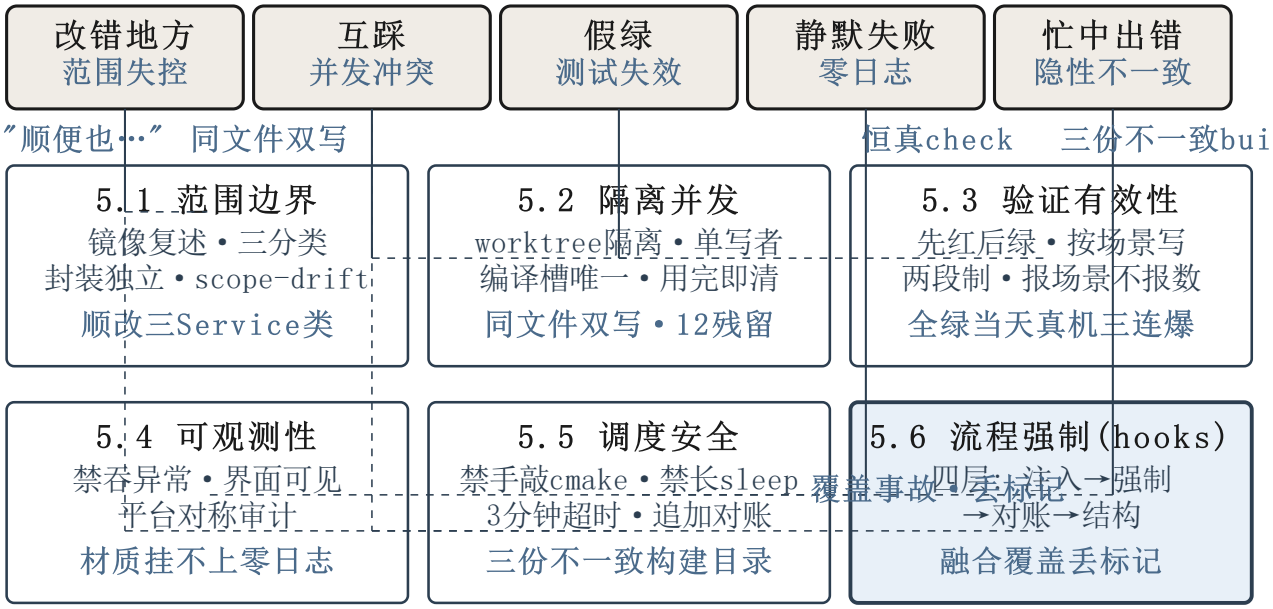
这个机制同样源自 rs-host 的经历。在一次 PDCA 多轮递升审计（解释：让 AI 多轮自动检查代码质量，每轮范围比上一轮广）中，审计员报了 10 项发现。用户逐一审过后拍板：只有 2 项是"当前需求交付物本身的缺陷"（功能没生效、路径不可达），剩下 8 项——包括历史遗留的空指针隐患、魔法数字没转枚举、注释风格不统一——全部标为"不修"。用户的原话是"千万别修改非需求的代码"。

此后形成了三分类法则：审计发现的每一条问题，必须归入三类之一——"需求交付物缺陷"（跟本次需求直接相关且功能错误，可修）、"本次引入但非需求"（是本次改动写出来的，但不是需求要求的东西，只记录不修）、"历史遗留"（`git log` 证明这个改动之前就存在，只记录并标 pre-existing 证据，不修）。修的范围只落在第一类，而且还要经用户确认。第二类的东西——比如顺手写的注释、顺手删的"看起来没用"的代码——最容易越界，因为它们"看起来无害"，但每一个看似无害的改动都在增加回归风险。这个机制把"要不要修"的决策权从 AI 手里拿回来，还给需求文档和用户。

表 5-1 收录（节内未展开条目）：

- **加字段封装独立方法**：给已有输出加字段时，新建独立方法，在既有装配方法末尾只加一行调用，不动核心逻辑。源自 rs-host 一次改动大量已有装配代码导致回归风险的事故，用户明确要求"封装独立方法，不动既有装配逻辑"。
- **scope-drift-audit 只审边界不审正确性**：审计范围外变更（"顺手多改了啥"）时，明确不管"逻辑对不对"——那是 code review 的事。源自审计子任务自行把"实现正确性"也纳入范围外审计导致误报的事件。

图5-1 预防机制全景图：五类风险 → 六组机制



图注：本图采用功能配色区分五类风险（灰底）与六组机制（白底/蓝底），非装饰用途。

图 5-1 预防机制全景图：五类风险到六组机制的映射

5.2 防"多线并行互相踩"（隔离与并发机制）

DevDesk 的开发模式是并行派发多个 AI 子任务同时工作——主会话拆分需求后，一次派出多个子代理各自到独立副本里写代码。但"并行"和"隔离"是两回事：光并行不隔离，结果是互相踩脚、谁都干不完。

条目三：worktree 隔离——每个子任务在独立的代码副本里工作【本项目】

worktree 是 Git 的一个机制（解释：从同一个仓库切出多个独立的工作目录，各自可以有不同的分支和未提交改动，互不可见）。DevDesk 把这条推到了铁律级别：凡是涉及编译或测试

的子任务，必须在 worktree 里开发，主工作树禁止跑构建。子任务在自己的 worktree 里完成改动、跑通测试、原子提交——整个过程和其他子任务完全隔离。

这个机制是真实事故直接撞出来的。项目早期，多个子任务在主工作树里同时写代码——一个改了 `settings_module.cpp` 还没提交，另一个也在改同一个文件的另一段，第三个在跑全量构建把工作区弄脏了。结果是"任务做完了但没法干净提交"——工作区里搅着三个人的改动，`git status` 一塌糊涂，哪个 commit 该含哪些文件完全分不清。最终只能强制清理工作区、根据子任务报告重建改动，三条线的时间成本全部白费。

worktree 隔离解决了这个问题的物理层面：每个子任务看到的是自己的文件系统，互不相干。但物理隔离只防"踩到别人的文件"，下一层问题是"踩到同一个文件"——这需要另一条机制。

条目四：同文件同时只有一个写者【本项目】

worktree 隔离之后，平行子任务各自有独立工作目录——但它们共享同一个 Git 仓库。如果两个子任务都改了同一个文件，合并的时候就会冲突。更糟的是，如果在飞的两个子任务被派发时没做交叠检查，它们各自在自己的 worktree 里改了同一个文件、各自提交、各自报完成——等主会话合并时才发现冲突，这时两条线的工作都已经完成了，回退一条就意味着那条线的全部工作量作废。

这个问题同样有真实事故。DevDesk 开发中曾有两个子任务同时被派发修改 `settings_module.cpp`——一个在做设置页的模块开关逻辑，另一个在做桌宠皮肤桥接。两条线各自在自己的 worktree 里改同一段区域，都正常提交了。合并时发现两边的改动撞在同一个位置，AI 判断不了该留哪个版本——只能人工裁决，代价是其中一条线的改动全部作废重做。

机制落地为一条硬规则：派单前先列出当前在飞的全部子任务、检查它们涉及的文件清单，如果新任务要动的文件有任何一个已经在某条在飞线上，新任务就排队等、不能派。规则不复杂，复杂的是执行纪律——派单时不能因为"快点开始"就跳过检查，漏一次就是一条线白干。

条目五：编译槽唯一——多线同时编译互相拖慢【本项目+方法论沿袭】

worktree 隔离让每个子任务有了自己的代码副本，但没有自己的 CPU 和磁盘。如果一个时刻有四个 worktree 在同时编译，四份构建任务抢同一块磁盘的读写、抢同一份编译缓存（ccache）的命中率——结果不是四倍产出，是四份任务都在等对方的 IO 完成。

DevDesk 实测过这个现象（2026 年 9 月）：在 12 核机器上全部放开了跑，四条 worktree 并发构建，单条耗时从原来的 40 秒涨到 2 分钟以上；ccache 的热命中率从九成以上掉到六成以下——因为每棵 worktree 的构建进程在交替写缓存文件，互相把热数据挤出缓存。这种互相拖慢不是"效率低一些"的问题——是会引发连锁反应：构建慢导致超时、超时导致子任务被判定卡死、误判卡死导致任务被终止重派。

机制落地为"编译槽"概念：无论同时有多少子任务在飞，同时只允许一个做编译/测试。编译槽被占时，其他需要构建的子任务排队等待。这个槽位由主会话统一管理——派构建类任务前先确认槽空闲，构建完成后释放槽。并行度封顶（默认每棵 worktree 最多 3 个编译并行单元）是另一方面——单棵独占了机器可以调高，多棵并存时必须压低，避免互拖。

表 5-1 收录（节内未展开条目）：

- worktree 用完即清：合并后立即删除隔离目录。曾一次清出 12 个残留 worktree，多会话长期并行下残留目录越堆越多，最后磁盘和 Git 状态都乱了。

5.3 防"假绿测试骗人"（验证有效性机制）

测试全绿不等于产品可用——这条道理谁都懂，但在 AI 驱动开发中，假绿的来源比传统开发更多样、也更隐蔽。DevDesk 在验证层面上撞过三堵墙：断言写法导致恒真（永远通过、永远不会失败）、测试覆盖偏离用户场景（测的是代码内部路径，不是用户走的路）、mock 环境掩盖真实世界的行为差异。

条目六：先红后绿——修复前必须证明测试会失败【方法论沿袭】

"先红后绿"（解释：先运行测试证明它确实失败了——红线；再修复代码让测试通过——绿线。如果一开始就是绿的，说明测试本身没写到点子上）是测试驱动开发的基本功。但在 AI 写测试的场景下，出现了一种更隐蔽的恒真断言：自定义测试框架里的 `check()` 函数，只打印失败信息但不抛出异常——结果测试跑完输出里写着 FAIL，但测试框架的"失败计数"永远是 0，报告显示"7/7 ALL PASSED"。

这个事故发生在兄弟项目的 cs-retro 工具开发中。它自研了一个迷你测试运行器，`check()` 内部用 `print` 报错但当次运行——不是写 bug，是造测试框架时漏掉了"失败必须 raise 异常"这条。后果：若干条测试在实际失败的状态下报告全绿，整个测试套件的信号完全失效。代码上线后真问题在线上暴露，往回追才发现"测试当时就是红的，只是没人看到"。

机制落地为两条：其一，自定义测试运行器必须保证失败＝非零退出码（让 CI 能拦住假绿）；其二，修复类改动提交前必须提供"先红后绿"证据——回退一次修复让测试变红、再应用修复让测试变绿。如果答不出"改动回退时哪条用例会红"，说明新增的断言是恒真的，不该提交。DevDesk 的直接消费方式：项目级 hook "commit 前先红后绿提醒"——这是一条提醒（不阻塞），但确实在每次提交时把纪律递到 AI 眼前（详见 5.6 节）。

条目七：按场景写测试——每条用例注明防什么具体事故【本项目】

这是 DevDesk 最惨痛的教训，也是若干机制中最直接来自用户纠正的一条。

2026 年 10 月 19 日，DevDesk 全量 1600 余条测试全部通过。CI 双平台绿。同一天，用户在自己的 Windows 真机上打开刚装的安装包，三分钟内连爆三个 bug：第一，从旧版本升级上来的用户数据库缺一个 `context_note` 列——创建任务时直接崩溃；第二，配置了公司内部模型网关跑智能体草稿生成，网关返回 400 错误——因为模型名里带了 `[1M]` 后缀，网关的实现不认识；第三，Windows 真机上的弹窗因为字体度量差异导致文字叠加、按钮出框。

这三个 bug 有一个共同点：每一个都在"代码世界的自洽"里是完美的——数据库迁移在全新安装时正常工作（旧库升级路径没人测）、mock 的模型端点永远不会拒 `[1M]`（真网关才拒）、无头几何断言在逻辑像素层面完全正确（但 Windows 真渲染的字体度量不同）。1600 余条测试全部守的是"代码内部路径"，没有一条守的是"用户走的路"——从旧版本升级、配真实网关、在真机上渲染。

用户当时的反馈直指要害："你已经写的单测，真的价值不大，只是个数字大。"此后测试方法论全面转向场景驱动：写测试前先列场景——这次改动交付给谁、他在什么环境上用、他从什么状态过来（升级前、新建、断网、不同网关）——每个场景一条用例，用例名写清防什么事故。测试目的不是"覆盖 XX 函数"，是"防用户从旧库升级时建不了任务"。场景缺口是单测数量补不了的——三个 bug 炸的时候 1600 条全绿，全绿与可用之间隔着存量数据库、真实网关、真机渲染三条缝。

条目八：智能体测试两段制——mock 之后必须真模型实测【本项目】

智能体类功能（任务执行、草稿生成、工具循环）涉及真实的 AI 模型调用——发给外部网关、等返回、解析响应。这类功能的测试天然棘手：如果全用 mock（解释：用假的模型端点替代真实网关，返回预设的假数据），测试跑得飞快，但永远碰不到"网关的返回格式跟文档写的不一样""网关拒绝某个你以为合法的参数""超时不是 30 秒而是 90 秒"这些现实问题。

DevDesk 在 2026 年 10 月 19 日的 UAT 验证中直接翻过一次车。在智能体草稿生成功能开发中，全部测试用 mock 的模型端点——发什么、回什么，完全在代码控制之中，所有用例全绿。但一接上公司内部模型网关真调，当场炸了：网关返回 400，因为请求里的模型名是内部网关专用模型名——方括号后缀是 Anthropic 协议的扩展标记，网关的实现不认识，直接拒。mock 端点永远不会触发这个问题，因为它不校验模型名字段——它只回预设数据。

此后立下两段制：第一阶段用 mock 端点验证理想路径和分支逻辑（速度、可控、不烧钱）；第二阶段必须用真实模型端点真调——草稿生成、任务运行、工具循环每类至少走一次真实调用。第二段不是走形式——mock 只证明"你的代码在理想世界能跑"，真实调用才证明"你的代码在我们用的那个真实网关上也行"。

条目九：测试绿不等于可用——交付口径从"报数字"改为"报场景清单"【本项目】

上面三条机制都指向同一个根问题：测试数量和通过率这两个数字，在一个不涉及真实环境的自动化流程里，对"产品能不能用"这个问题的回答力几乎为零。但 AI 天然倾向于报数字——"1600+ 条全绿"听起来比"Markdown 笔记从创建到搜索的产品级完整流程还没跑过真窗口探针"要确定得多。

自 2026 年 10 月 19 日起，DevDesk 的交付口径从"报数字"改为"报场景清单"：每次交付汇报不写 Total/Passed/Failed，只写三行——哪些业务场景实际跑起来了（真窗口/真网/真机）、结果如何、哪些没跑到（明写"未验证"）。交付前三问（存量数据升级路径验过吗、真实外网/网关验过吗、目标平台真机验过吗）作为发任何安装包前的固定问答——答不上来的写进"未验证"栏，不许沉默。

这个机制本身不修任何 bug。它修的是一个更危险的东西：给用户虚假的安全感——"测试全绿"四个字会让人以为系统是可靠的，而实际上三条大缝全在测试的视野之外。用场景清单替代数字汇总，不是为了写得更好看——是为了让"没测到"这件事变得可见。

5.4 防"静默失败无人知"（可观测性机制）

代码不出错的最好证据不是"看起来对"，是"出错了一定看得见"。静默失败（解释：程序内部发生了错误但不报任何日志、不崩也不响，看起来一切正常，实际某个功能根本没生效）是 C++ 桌面应用中最难排查的 bug 类型——开发时看不出来，用户用的时候也看不出来，只能靠"某个功能就是不对劲"的模糊感觉去猜。

条目十：禁止静默吞异常——兜底逻辑必须留至少一行日志【本项目】

DevDesk 开发中出过一次典型事故：应用的主题材质切换功能写好了，代码逻辑看起来全对——切换主题时调取新配色、挂到界面组件上——但实际切主题后界面完全没有变化。排查了很久才发现根因：材质加载函数里有一个 `try-catch` 块，catch 分支写的是 `return false`——捕获了异常，返回了 false，但一行日志都没有。调用方看到 false 就退出了，也没有日志。结果就是：材质从未挂上过，但整个调用链上所有环节都"正常退出"了，没有一行错误信息。

机制落地为硬规则：任何兜底分支——catch 块、默认值、clamp 到下限/上限、静默跳过——至少留一行日志，写清"什么操作、在哪失败了、为什么被兜底"。这不是性能优化，是事后排查的唯一线索。在 C++ 桌面应用里，没有日志的失败和没有发生过的失败没有区别——时间一过，现场消失，只剩下"这个功能有时候会不对"的模糊反馈。

表 5-1 收录（节内未展开条目）：

- **撞闸和失败原因界面可见**：智能体任务"没跑起来"或"跑了但失败"时，失败原因必须在界面上能直接看到。不埋在二级菜单或日志文件里——用户不需要学 grep 也能知道发生了什么。

- **平台对称性审计**：改动只在一个平台上验证过，另一平台可能当场崩。DevDesk 在 Windows 开发期间写的 macOS 代码从未编译过——回到 macOS 后首次构建 115/117 条用例失败。此后每个改动的审计必问一句："另一平台的代码路径会不会因此变化？"

5.5 防"忙中出错还看不见"（调度与安全机制）

并行开发的效率上限不取决于"同时能跑多少任务"，而取决于"出错时多快能发现"。在一个多子任务并发的系统中，最常见的两类事故是：构建命令的"隐性不一致"（看起来都跑通了，但构建目录互相打架）和长时间等待掩盖的"假死"（进程其实已经卡住了，因为超时设得太长所以没人发现）。

条目十一：禁止手敲 cmake 构建——一切走封装脚本【本项目】

DevDesk 的 C++ 编译走 CMake 构建系统。CMake 命令有大量参数——构建类型（Debug/Release）、编译缓存开关、框架路径、加密库路径、并行度上限——手敲一次不漏已经很考验注意力，在不同的 worktree 里反复手敲，几乎必然出现参数不一致。

2026 年 9 月 28 日，开发机器上同时存在三个构建目录——`build/`、`build-release/`、`build-debug/`——分别是之前不同时间手敲 `cmake -B build ...` 造出来的，参数各不相同。其中一个缺 OpenSSL 路径（configure 报 Warning 而非 Error，悄悄降级了加密能力），另一个没开 ccache 编译缓存（每次全量重编），第三个连并行度参数都没设。三个目录互相独立、互不可见，没人知道哪个是最新正确的构建结果。

这次事故的直接后果是"构建成功但测试结果不一致"——同一个测试在 build/ 里过、在 build-release/ 里红，原因不是代码有问题，是两个目录的编译配置不一样。排查浪费了一个多小时，最终才找到根因：三条手敲的命令表面上都"跑通了"，实际上不是同一件事。

机制落地为一条硬规则，写进项目核心文档：构建和测试一律走封装脚本 `scripts/build-test-run.sh`，禁止手敲 `cmake -B` 和 `cmake --build`。脚本内部处理工具链路径、平台参数、编译缓存开关、并行度封顶——在每一个 worktree 里输出的构建目录是同一套配置。脚本是唯一被反复执行的构建路径——CI 用它、worktree 用它、主会话用它。不允许存在"我这次手敲的也差不多"——差不多就是不一致，不一致就会在某个时刻以莫名其妙的结果出现。

表 5-1 收录（节内未展开条目）：

- **禁止长 sleep（单次不超过 60 秒）**：长时间固定等待掩盖进程卡死——等 120 秒后超时退出和等了 60 秒发现没动静再排查，后者成本更低。
- **Bash 超时上限 3 分钟**：长构建放后台跑 + 完成通知，不靠前台长等。超长 timeout 掩盖卡死——把 10 分钟顶满当默认等于放弃卡死检测。

- **追加消息对账**：对在飞子代理追加任务有概率被静默忽略——消息投递与代理收尾存在竞态。机制：追加即登记，完成时逐项对账，缺项自动补做。详见第 7 章（AI 行为监控——运行时关卡层）。

5.6 防"AI 不走流程"（流程强制机制——hooks 体系）【本项目】

前面五节的机制有一个共同的脆弱点：它们都是"规则"——规则需要被记得、被遵守。而 AI 会话天然不记得上一会话的上下文——过去的纪律写在文件里，但文件如果太长、AI 可能选择性忽略；文件如果太短、关键判断点可能没写进去。口头嘱咐"别忘了先审计再测试"等于没嘱咐——下一条 AI 会话不知道你说的这句话。

2026 年 10 月 21 日，DevDesk 引入了一套把"规则"升级为"机器闸门"的机制——不只是把纪律写在文档里等 AI 读，而是在 AI 执行动作的当下由脚本拦截和判断。这套体系按保证强度从低到高分四层。

四层机制的递进逻辑

第一层：注入层——把规则写进每次 AI 会话启动时自动加载的文件。项目级 CLAUDE.md 和九个跨项目经验文件（memory/01~09）由 `@import` 指令在会话启动时全量注入——AI 在动手之前已经被塞进所有已知坑。这一层的保证是"AI 知道了规则"——但没有强制力。AI 可以选择性地遵守或忘记，尤其是在长会话的上下文被大量 token 冲淡之后。

第二层：强制层——hooks 脚本与守卫，判定权在机器，AI 绕不过。这是新引入的一层，由四条项目级 hook 组成（见下文）。它的核心设计取舍是：提醒不阻塞（常响的提醒等于没有提醒），守卫才拦截——强制力只花在最贵的事故类型上。hook 脚本本身是纯 shell 实现，不依赖 jq 等外部工具（hook 运行环境不保证用户 PATH），静默失败的风险通过"依赖必检 + 缺件告警不阻塞"的 fail-soft 策略控制。

第三层：对账审计层——独立于实现者的 AI 会话做事后检查。这一层的保证来自"不同 AI 之间的互相验证"：审计员不能是实现者本人（第 4 章阶段 2），同一事实需要两次独立证实才能采信（第 7 章反幻觉机制），追加消息完成后的逐项核对（第 7 章运行时关卡）。对账层比注入层强——不止是"知道规则"，还有"第二个人在检查"。

第四层：结构层——让正路比歪路更省力。构建走一条脚本（见 5.5 节条目十一），文档走唯一入口（所有 spec 从同一个路线图派生），worktree 的创建和清理由脚本自动管理。这一层的保证不靠"人记得"，靠"走歪路更费劲"——AI 天然选择阻力最小的路径，如果正路被封装成一条命令、岔路要手敲六个参数，AI 会自己走到正路上。

这四层不是一次设计出来的。注入层和结构层是最早有的——CLAUDE.md 自动加载和构建脚本封装在项目第一天就立了。对账审计层是随着六阶段流程（第 4 章）逐步完善的——每次审计

发现"流程没人跟"就补一条检查。强制层是 2026 年 10 月 21 日才引入的，直接触发事件是一次"多轮融合覆盖事故"：后一轮对基线文件的改写挤掉了前一轮注入的内容（每轮只验本轮、全绿仍漏丢），靠事后对账才发现历史上丢掉的标记。这个事故证明了注入层不够——当改动者自己就是"审核者"且只审核当轮，历史内容丢失无法被发现。

四条项目级 hook（强制层最小实现）

hook 一：基线内容守卫（唯一会阻塞的一条）。当 `docs/strategy/`、`docs/adr/`、`docs/superpowers/`、`GLOSSARY.md`、`CLAUDE.md` 这些战略基线文件发生改动时，自动触发累积核对——跑一个脚本，逐项回归一份"累积清单"里的全部历史交付标记（每一行是一个"文件路径 + 期望存在的关键字"），任何一条标记在当前文件中不存在则脚本退出码非零，拦下这次编辑请求并列出缺失项。五一遗漏则这次改动就被判定为"挤掉了历史内容"——要么恢复内容，要么明确声明有意废弃并更新清单、注明依据。这条 hook 不常响——只在基线文件被改动时触发，且只在内容真丢了时才拦截。设计原则是"异常时才说话，正常时静默"。

hook 二：合并后门禁提醒。当执行 `git merge`、`git pull`、`git rebase` 时触发——打印一句"分支绿不等于合并绿"，并附真实事故的提交编号（一次合并进主分支后从未构建，当场无法编译）。这条 hook 不阻塞，只递到眼前——使用者看到提醒后决定要不要立刻跑一次构建。

hook 三：提交前"先红后绿"提醒。当执行 `git commit` 时触发——检查本次提交是否含行为变更（非纯文档/配置），若是，打印一句"先红后绿"——提醒在提交前确认有测试先红后绿的证据。恒真断言等于假绿。不阻塞，只提醒。

hook 四：改源码前"plan 先行"提醒。当 Edit/Write 工具目标路径命中 `src/` 时触发——提醒非平凡改动须先有设计文档或开发计划作为依据，否则"无锚点审计"。一行修复可豁免，但需自证 trivial。不阻塞，只提醒。

这四条 hook 的实战验证在上线当天就发生了一次自我订正。hook 配置文件本身需要 AI 会话启动时装载——但首次部署时，会话在配置文件创建之前已经启动了，装不进去。机制本身的"装载时机"缺陷被当场发现并记录：会话启动后新建的配置文件本会话不装载、需重启生效。换句话说，这套机制上线后的第一个"bug"是它自己装不上自己——发现后立即订正了装载时机规则。这恰好是一个活生生的例证：机制和人一样，第一次上线不可能完美，但只要"发现→订正"的闭环在转，它就会越来越好。

图5-2 四层机制示意图：从"规则在纸上"到"规则在闸上"

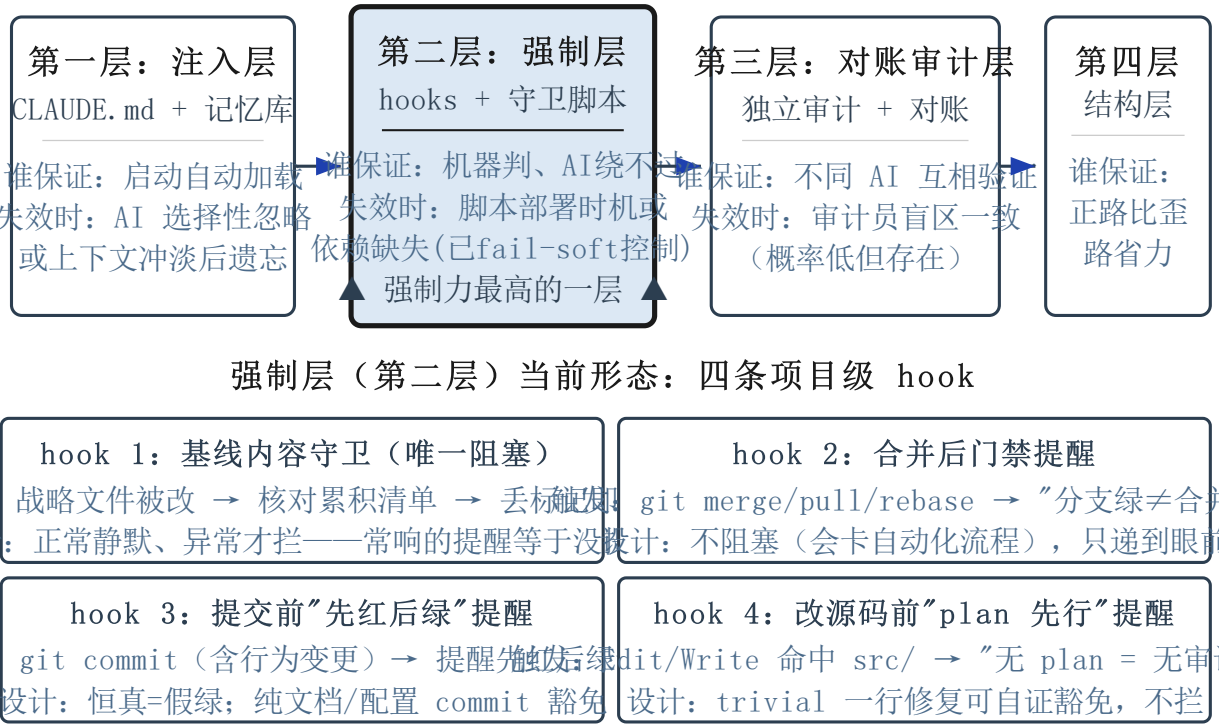


图 5-2 四层机制示意图：从“规则在纸上”到“规则在闸上”

表 5-1：机制-事故对照表

机制名	防什么	代表事故	事故来源	引入日期
镜像复述	AI 自扩展范围，把单点操作变成批量修改	AI 因"既然改了 A 顺便 B 也"把同目录三个 Service 类的同名字段一起改了	方法论沿袭（rs-host）	经验库沉淀期
只修需求交付物缺陷	审计发现全修导致范围失控	PDCA 审计 10 项发现中 8 项被用户标"不修"——包括历史 NPE、魔法数字、注释风格	方法论沿袭（rs-host）	经验库沉淀期
加字段封装独立方法	给既有装配逻辑加字段导致回归	改动大量已有装配代码，用户明确要求"封装独立方法、不动既有装配逻辑"	方法论沿袭（rs-host）	经验库沉淀期
scope-drift-audit 只审边界	范围外审计把"实现正确性"当范围问题误报	审计子任务自行把"匹配键对不对"当范围外变更上报，用户连说"不要管和需求无关东西"	方法论沿袭（rs-host）	经验库沉淀期
worktree 隔离	多子任务同目录写码互相弄脏工作区	三个子任务同时改主工作树，`git status` 一塌糊涂，哪个 commit 该含哪些文件完全分不清	本项目	2026-09
同文件同时只有一个写者	两个子任务改同一个文件，合并冲突导致废掉一整条线	两个 agent 同写 `settings_module.cpp`，只能终止其中一条线、改动全废重做	本项目	2026-09
编译槽唯一	多线并发编译互拖 IO 和 ccache 命中率	四棵 worktree 同时编，单条从 40s 涨到 2min+，ccache 命中率从 90%+ 掉到 60%	本项目+沿袭	2026-09
worktree 用完即清	残留 worktree 越堆越多，磁盘和 Git 状态乱	一次清出 12 个残留 worktree	本项目	2026-09
先红后绿	恒真断言导致假绿测试	自定义 check 只 print 不 raise，失败计数永远 0，"7/7 ALL PASSED"全是假象	方法论沿袭（cs-retro）	经验库沉淀期
按场景写测试	测试覆盖代码路径不覆盖用户场景	全量 1600+ 条全绿当天用户真机三连爆——老库缺列崩溃、网关拒模型名、弹窗叠加	本项目	2026-10

智能体测试两段制	mock 全绿掩盖真网关行为差异	mock 全绿但公司内部模型网关拒带方括号后缀的模型名，草稿生成当场 400	本项目	2026-10
测试绿不等于可用	数字"全绿"给用户虚假安全感	同上真机三连爆事故——全绿与可用之间隔着存量库、真网关、真渲染三条缝	本项目	2026-10
禁止静默吞异常	兜底 return false/clamp 不报日志，事后零线索	材质从未挂上过却一行日志都没有——catch 块只有 `return false`	本项目	2026-09
撞闸和失败原因界面可见	智能体失败原因埋在日志里，用户不知道发生了什么	智能体任务"跑了但没结果"，用户需要排查才知道是权限问题	本项目	2026-10
平台对称性审计	只验单平台，另一平台当场崩	Windows 开发期间写的 macOS 代码从未编译，回 macOS 后 115/117 条失败	本项目	2026-09
禁止手敲 cmake	多份构建目录参数不一致，测试结果互相矛盾	机器上同时存在 build/build-release/build-debug 三份不一致的构建目录	本项目	2026-09
禁止长 sleep	长时间固定等待掩盖进程卡死	兄弟项目（monitor）中 sleep 120s 让卡死进程多挂了一倍时间才被发现	方法论沿袭	经验库沉淀期
Bash 超时上限 3 分钟	超长 timeout 掩盖卡死，把 10 分钟顶满当默认等于放弃卡死检测	2026-10-02，子任务在后台卡死数十分钟才被发现，timeout 设得太高完全不起作用	本项目	2026-10
追加消息对账	追加任务被静默忽略	UAT6 中追加任务被静默忽略，靠事后对账才追回（详见第7章）	本项目	2026-10-21
四层机制 + 项目级 hooks	规则只在文档里等 AI 读，没有强制力	多轮融合覆盖事故——后轮改写挤掉前轮注入内容，每轮只验本轮、全绿仍漏丢	本项目	2026-10

本章的 12 条完整叙述和表 5-1 的 20 条机制，共同回答了第 1 章提出的"怎么做到的"的核心问题：不是 AI 本身有多聪明——是每一次事故之后，不是去骂 AI，而是去问"什么样的机制能让这个事故以后不会再发生"。答案有时候是一行日志、有时候是一条 hook 脚本、有时候是整套交付口径的翻转——但它们的共同点是：把"人的记性"替换为"机器的判定"，把"下次记住"升级为"下次绕不过"。下一章将讲述这些机制的上游——事故发生之后，怎样把它从"一个人踩过的坑"变成"所有后续 AI 会话都知道的规矩"。

第6章 自动复盘：事故如何沉淀为经验

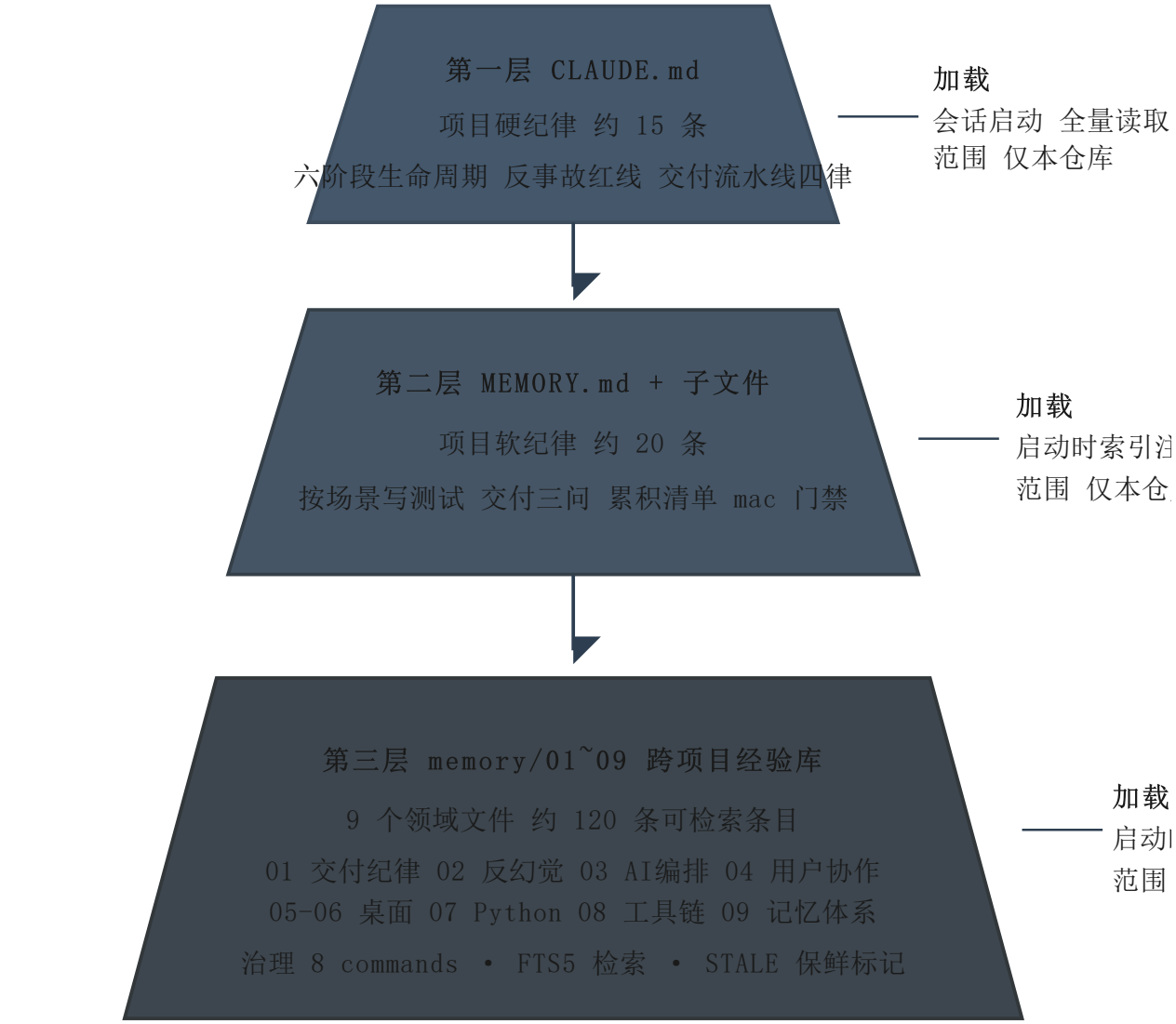
如果开发经验只存在人的脑子里，人一走就丢；如果只在文档里，没人翻就是废纸。DevDesk 开发一个月后，它的知识总量——代码架构、设计决策、事故教训、行为纪律——已经超过了任何一个参与者能全局掌握的程度。但项目仍然稳定运转，因为经验不在人的脑子里，在一个"每次 AI 启动都自动加载"的位置。它不是用来查的——是用来每次干活前先当规矩读一遍的。

本章讲三件事：经验存在哪里、怎么从事故变成经验、以及一条经验从诞生到永续生效的完整生命周期。

6.1 经验的三层存储结构

DevDesk 的经验不是堆在一个文件里的，它按"生效范围从窄到广、加载频率从高到低"分了三层（图 6-2）。

图6-2 经验的三层存储结构（倒金字塔）



图注：倒金字塔上窄下宽，代表"条目数量由少到多、生效范围由窄到广"。三层在会话启动时

图 6-2 经验的三层存储结构（倒金字塔）

第一层：项目硬纪律——CLAUDE.md

这是 DevDesk 仓库根目录下的一个文件。每次 AI 会话在 DevDesk 仓库里启动时，Claude Code 会自动读取并遵守这个文件里写的所有规则。里面装的都是"违反就出事"的硬纪律：六阶段开发生命周期（实现、审计、修、测试、合并、验收一步不能跳）、交付流水线四律（只跑定向测试、worktree 隔离、完成即合并、用完即清）、反事故红线（禁止批量清理在飞的 worktree、禁止跨仓库改动不走补丁流程）、以及"已踩过的坑"清单（单位错配、三套坐标系、自定义检查只打印不报错就是假绿）。

这一层的经验有三个特点。一是"每次启动都读"——它不是参考手册，是行为边界。AI 在动手写任何代码之前已经知道了所有已知的坑。二是"门槛最高"——只有反复发生、后果严重、且有明确实证事故支撑的经验才能进这层，不是出一次小问题就往里加东西。三是"加载时机最早"——在 AI 开始思考"怎么做"之前，它已经知道"不能怎么做"。

第二层：项目软纪律——MEMORY.md

CLAUDE.md 不能无限膨胀——把一百个细碎的备忘全塞进去，AI 反而会迷失在噪音里、忽略真正重要的硬纪律。因此 DevDesk 把"项目独有但粒度更细"的经验放到第二层：项目记忆文件。它不直接装规则正文，而是做索引——一行标题概括一条经验，指向一个单独的详细文件。

这一层的条目包括：子任务并发上限从 4 升到 8（2026 年 10 月 21 日用户上调）、修复类任务先改码再统一验证（不每个子任务自己跑测试拖慢进度）、按场景写测试每条注明防什么事故（真机 UAT 三连爆的教训）、所有业务场景必须在真窗口里实际跑过才算数（无头单测绿不算"实际跑过"）、交付时不报测试数字报场景清单和未验证清单、智能体类测试必须经过两段——先 mock 验证理想路径再用真模型实测收口、同文件多轮派单后必须做累积清单回归核对（后轮覆盖前轮的教训）。

第二层的加载方式与第一层不同：CLAUDE.md 是启动时全文注入，MEMORY.md 是索引注入——AI 知道有哪些条目，需要细节时去读子文件。这种设计让硬纪律始终聚焦，细颗粒度的经验也不丢失。

第三层：跨项目经验库——memory/01~09

这是最底、最广的一层。九个按领域拆分的文件（交付纪律、反幻觉、AI 编排、用户协作、Windows 桌面、前端桌面、Python 脚本、工具链环境、记忆体系），总共约 120 条可检索的经验条目，覆盖了 DevDesk 开发中可能遇到的大部分非业务问题。

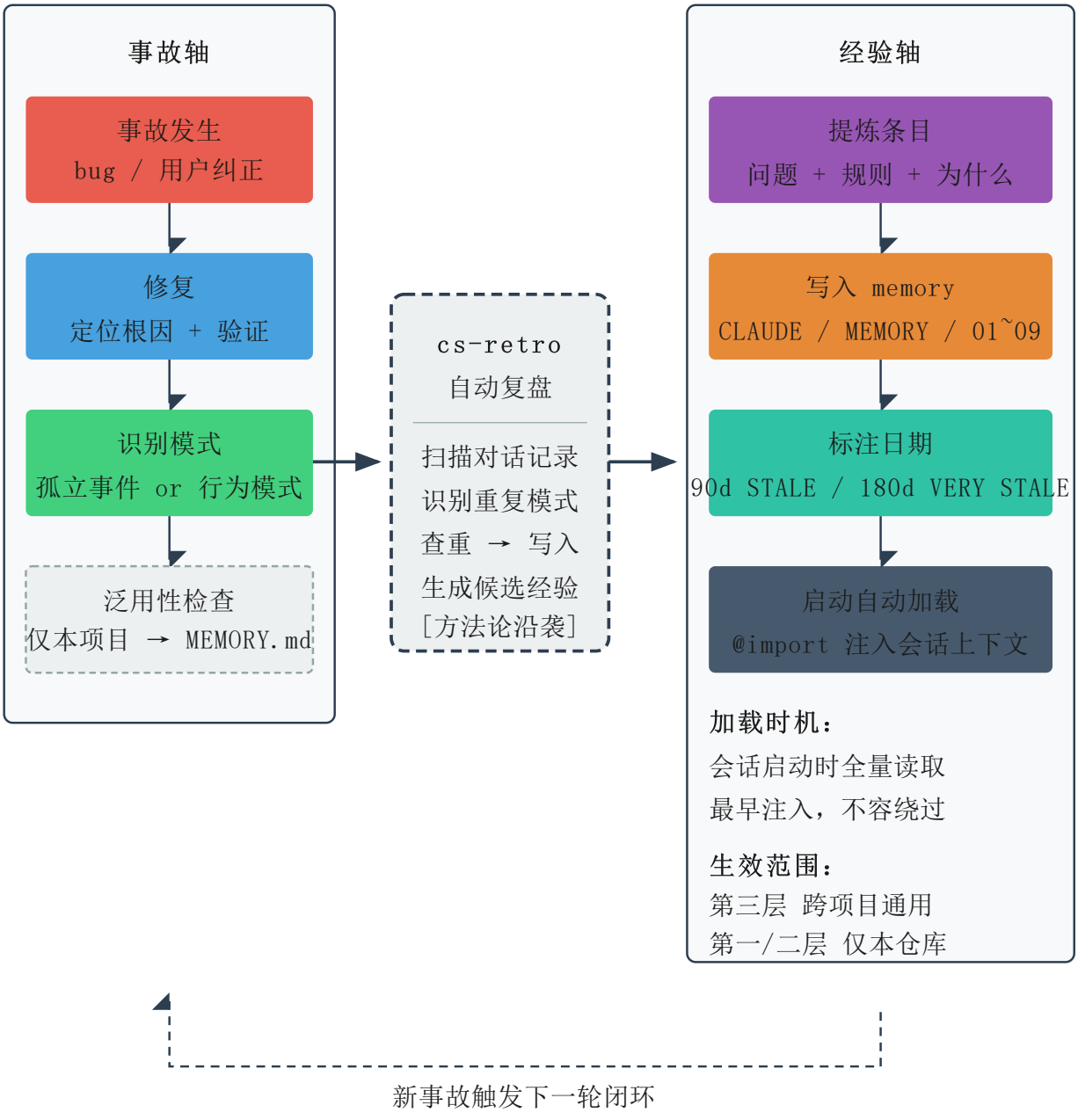
这一层的经验不来自 DevDesk 一个项目。大部分条目来自多个项目的共同教训——rs-host（Java 后端）、winui3（Windows 桌面应用）、monitor（系统监控工具）、Tauri/Rust 桌面项目——每个项目出的事故提炼成本，写进跨项目经验库，后续所有项目共同受益。DevDesk 继承这套经验库时已经包含约九十个条目，是一个月的密集开发中几乎没有重复踩坑的重要原因。

它的加载方式与 CLAUDE.md 完全一致：每次 AI 会话启动时，用户级的 `~/.claude/CLAUDE.md` 通过 `@import` 指令把这九个文件全文注入。对 AI 来说，项目独有规则和跨项目通用经验在加载后没有区别——都是"动手前就要知道的东西"。但对维护者来说，三层分治意味着：改一条 DevDesk 特有的经验不用翻跨项目文件，改一条通用经验也不会污染 DevDesk 的 CLAUDE.md。

6.2 经验闭环：从事故到永续生效

经验不会凭空出现。DevDesk 的每一条经验都是一次事故"撞"出来的。从事故发生到经验永久生效，有一条固定的流转路径，可以概括为四个环节（图 6-1）。

图6-1 事故到经验的闭环流程



图注：左侧为事故产生至识别模式的单向推进；右侧为经验提炼至激活的持续循环；中间虚线左侧底部的泛用性检查决定经验进入项目层（一/二）还是跨项目层（三）。底部虚线回路表

图注：本图采用功能配色区分事故轴（左）与经验轴（右），非装饰用途。

图 6-1 事故到经验的闭环流程

触发：识别"这是模式，不是孤例"

不是每个 bug 都值得变成一条通用经验。区分标准是看它的根源是"单点错误"还是"行为模式"——如果只是某个函数把参数传反了，修完就完了，不需要提炼纪律；但如果它是"AI 在类似场景下会系统性犯同一类错误"，比如每次看到"看起来重复"的东西就想顺手清理、每次子任务报告"完成了"但实际上文件没落盘，那就不是单点 bug——是一种需要永久性防范的行为模式。识别这一步通常由 AI 自己在复盘时完成，也可能由用户一句话点出——"这类问题已经是第 N 次了"。

沉淀：从事故到纪律到文件

一旦识别出模式，下一步就是把它提炼为一条纪律。提炼有四个要素：问题是什么（现象描述，让 AI 能识别自己是不是正在犯同样的错）、规则是什么（可执行的行动约束，不是空洞的"以后注意"）、为什么（一句话链接到原始事故，防止将来有人觉得"这条是不是太严了"想删掉）、写入哪里（按主题选对应的文件——交付纪律进 01、反幻觉进 02、AI 编排进 03）。

沉淀时要过两道检查。查重：同一条规则有没有已经写过但表述不同？有就合并，不新建——同样的规则换三句话写三遍，只会让 AI 困惑"这三条是不是在说同一件事"。泛用性检查：这条规则是只有 DevDesk 能用，还是换个项目也有用？只有 DevDesk 能用的放第二层 MEMORY.md，通用的进第三层跨项目经验库。

以"测试绿不等于可用"这条经验为例（完整档案见表 6-1 第 1 行）：事故发生在 2026 年 10 月 19 日，用户真机 UAT 三连爆——存量数据库缺少新字段导致建不了任务、真实网关拒绝带特殊标记的模型名称、Windows 字体度量差异导致弹窗文字叠加。当天全量 1600 多条测试全部通过，但三个 bug 全在测试覆盖的缝里。沉淀时分了三步：先把"测试绿不等于可用"写进 CLAUDE.md 的验证原则节（硬纪律），再把"按场景写测试"和"交付不报数字报场景清单"写进 MEMORY.md 子文件（软纪律），最后把"所有业务场景必须实际跑起来"也独立成条目。一条事故拆成三层对应——不是多余，是每层读者不同：CLAUDE.md 的读者是 AI 实现者，需要知道"测试过了不算完"；MEMORY.md 的读者是 AI 编排者，需要知道"派测试任务时要怎么派、交付报告要怎么报"。

保鲜：经验也需要防过期

写进文件不是终点。技术环境会变、项目阶段会变、工具链会换——一条经验放久了可能失效。DevDesk 的经验体系用了一个简单但有效的方法：每条经验的标题后面挂着 `last_verified`（最晚验证日期）。日期超过 90 天标为"可能过时"，超过 180 天标为"很可能过时"——在 AI 读取时会提示复核这条是否还适用。如果一条纪律被连续违反三次以上（说明它要么不切实际、要么没人看了），就会进入"考虑删除"的队列。

保鲜不是一次性的年终大扫除——它有专门的一套自动检查工具（`/memory-audit`），定期扫描全部经验条目，按日期、重复度、索引断链三维打分，产出一份"哪些条目需要人工决策"的报

告。报告本身是只读的——不自动删、不自动改——所有修改动作都需要人确认。

激活：启动即注入，不问不查

前三步（触发、沉淀、保鲜）保证了经验的"质量"。但质量再高，如果 AI 每次干活前不读它，就是废纸。激活这一环才是整个机制的最后一公里——经验必须被加载到 AI 的启动上下文里，才能真正影响行为。

激活机制的实现方式很直接：Claude Code 在每次新会话开始时，会自动读取用户配置的 CLAUDE.md 文件。用户级 `~/.claude/CLAUDE.md` 里用 `@import` 指令声明了九个 memory 文件，所以这九个文件在会话启动时就被全文注入了 AI 的上下文——它在开始回答任何问题之前，脑子里已经装满了所有已知的坑。这相当于每次换班时，下一班的工人拿到的不只是一句口头交接，而是一本写满"这些事情不能做、这些地方容易出问题"的完整手册。

从事故到永续生效的这四个环节——触发、沉淀、保鲜、激活——在 DevDesk 的一个月开发周期里运转了至少二十次。每运转一次，系统的"经验总量"就增加一条，而下一次AI干活时的"已知坑清单"就多一行。

6.3 cs-retro：自动复盘的辅助机制【方法论沿袭】

前两节讲的经验闭环主要靠"手动触发"——事故发生后，由人（或人指令 AI）识别模式、提炼纪律、写入文件。但还有一条辅助路径：让 AI 定期自己翻自己的对话记录，主动找出"值得沉淀成经验"的模式。这套机制叫 cs-retro，它不来自 DevDesk，而是在兄弟项目中逐步演进出来后被 DevDesk 继承使用的工具。

它的工作原理可以概括为四步。第一步是扫描：定期读取最近的 AI 对话记录，从中找出两类信号——重复出现的问题模式（比如"不同日期的多个对话里 AI 都在犯同类错误"）和被用户明确纠正过的错误（用户的纠正本身就是最珍贵的经验素材）。第二步是提炼：把扫描到的信号抽象成候选经验条目——去掉项目专有名词、去掉具体的参数值、提炼出"在什么情况下 AI 会犯什么错、应该怎么防"的行为模式。第三步是查重与泛用性检查：这条候选经验是不是已经写过了？换个项目也有用吗？只有通过这两道检查的候选才能进入下一步。第四步是写入：向用户展示预览，确认后写入对应领域的 memory 文件。

cs-retro 作为方法论，在 DevDesk 中主要是"借来用"而非"从头造"——它的核心实现（对话记录扫描、模式识别、查重引擎）由配套工具侧提供，DevDesk 侧的角色是"经验消费者"：启动时通过 CLAUDE.md 和 memory 文件的 `@import` 机制全额加载跨项目经验。本章的实证重心因此在 DevDesk 自身的"事故到经验"闭环——那些在这个仓库里真实发生、被提炼、被写入、被激活、最终改变了后续 AI 行为的具体案例。

6.4 四个生命周期案例：从事故到永续规则

下面四条经验都源自 DevDesk 自身的真实事故。每一条的完整生命周期——从事故发生到纪律写入到被自动加载——都可以在这个仓库的 git 历史和文件里找到对应证据。

案例一："测试绿不等于可用"（2026 年 10 月 19 日）

2026 年 10 月 19 日，DevDesk 全量 1600 多条测试全部通过。同日，用户在自己的 Windows 真机上安装新版本后，连续爆出三个问题——完整细节与实证分析见第5章条目七。三个问题全是"用户从旧版本升级"或"用户配真实网关"这类场景下的问题——而现有测试没有一条覆盖这些场景。

这次事故触发了 DevDesk 测试方法论的一次根本转向。当天，用户明确指令"测试要按场景写，每条用例都有具体的目的"——从"覆盖 XX 函数"变成"用户在什么状态下、在什么环境里、做什么操作"。同一天，"交付前三问"也立了起来：发任何包之前必须回答——存量数据升级路径验过吗？真实外网网关验过吗？目标平台真机验过吗？答不上的必须在报告里写"未验证"。

这组事故的沉淀路径是三层同步：硬纪律写进了 CLAUDE.md 的验证原则节（"测试绿不等于可用""交付前三问"），软纪律拆成了四条 MEMORY.md 子条目（按场景写测试、所有业务场景必须实际跑起来、交付不报数字报场景清单、修复合并不等于交付），通用经验则已隐含在跨项目经验库 02 号文件（反幻觉）的"自定义 check 必须 raise"等条目中。截至 2026 年 10 月 21 日，这组规则的 `last_verified` 标为 2026-10-19，仍在活跃生效。

案例二："追加消息必须对账"（2026 年 10 月 21 日）

DevDesk 的开发模式是主会话一次派出多个子任务并行工作。有时主会话在子任务已经起飞后想到还要追加一条要求，就会给在飞的子代理发一条追加消息。但投递机制存在一个竞态：消息在代理的"下一个工具轮"才投递，如果代理已经进入收尾阶段（最后一个工具轮已过），追加消息就会被静默吞掉——代理的正常完成通知照样发回来，而追加的那件事根本没做。

追加消息对账机制的完整叙述见第7章 7.2.3（UAT6 两起遗漏实证），本节仅交代其经验沉淀路径——

从这个事故提炼出的纪律（"追加即登记、批报告逐项对账、临近完成的批不再追加"）已被写进跨项目经验库 03 号文件（AI 编排），作为一个模式条目永久生效。它的生效机制是正向的：AI 在准备追加消息时，会先检查目标代理是否已经进入收尾——如果是，追加内容自动滚进下一批的冻结清单，不在本批追加。项目级在 CLAUDE.md 的"项目级 hooks"条和"交付流水线"规则中已有提及，作为强制层对账守卫的背景依据。

案例三："主会话只编排不实施"（2026 年 9 月 27/28 日，2026 年 10 月 21 日再强化）

项目早期，主 AI 会话既拆需求、又派子任务、又亲自改代码。2026 年 9 月 28 日，这种模式的问题集中暴露：主会话连续直接修改源码和配置文件，导致"既当球员又当裁判"——审代码的时候看的是自己刚写的东西，天然缺乏第二方视角，多个逻辑错误直接通过了。用户当天定下铁律：源码、测试、配置、文档一律派子任务改，主会话只做编排和把关。

但"知道规则"不等于"始终遵守"。2026 年 10 月 21 日，主会话再次踩线——把项目级 hooks 的落地当成"配置任务"，自己动手写了 `settings.json` 和两个 hook 脚本。用户当场纠正："不要再主会话执行开发任务"。教训进一步细化：即使某个 skill 的文档里写着"可以直接 Read 后 Edit"，进了 DevDesk 的"一切落盘文件改动一律派子任务"的铁律范围，项目纪律优先于通用指引。

这条经验的生命周期展示了一个模式被反复强化后逐渐"硬化"的过程。第一版纪律（2026 年 9 月 28 日）只是 CLAUDE.md 里的一条规则；第一次违反（2026 年 10 月 21 日）催生了 MEMORY.md 子条目 `main-session-no-implementation.md`，并把"再犯记录"也写进去作为实时警示；与此同时，同日的累积核对清单强制化（案例四）从另一个方向堵住了"主会话手改文件"可能导致的遗漏——就算主会话碰了文件，累积清单守卫也会在内容被改坏时拦截。三层机制从不同角度锁住了同一类风险。

案例四：累积核对清单的诞生与强制化（2026 年 10 月 21 日，当日全链闭环）

这是 DevDesk 经验沉淀体系中最完整的一个闭环案例——从事故到机器强制到自我订正，整个链条在一天内走完。

2026 年 10 月 21 日，DevDesk 正在进行战略基线文档的多轮融合：将前期调研产出的研究成果注入项目文档。因为是同一份文件（`05-技术路线图.md`）需要连续四轮派单注入，每轮由不同的子代理负责。

前三轮平稳推进。第四轮注入完成后，主会话按惯例验收：grep 确认"本轮交付项"全部在文件里——结果全绿。但独立的代码审计在"遗漏检测"维度上发现了一个惊人事实：第四轮和第三轮的改写条目时，把第一轮注入的六个条目（包括 `outcome` 字段、四条已保存规则、进度回调说明、方案分叉说明、源码角色参考）加上总览索引的一行标记——一共七个标记——全部挤掉了。单轮验证全绿，但累积内容丢了三分之一，无人察觉。

这个事故的根因不是某个子代理写错了——每个子代理都在按自己的 prompt 认真干活。根因是验收方法本身的漏洞：每轮只验证"本轮交付项"，不验证"全部历史轮次的累积交付项"的完整性。后轮改写某个段落时把前轮注入的标记覆盖掉，这个覆盖行为在本轮验收的 grep 条件下是不可见的。

事故发生后，当天立了三条规则。第一，同文件多轮派单后，验收不是"本轮交付项在"——是"全部历史轮次交付项的累积清单逐项 grep 回归"。第二，给后续轮次的 prompt 必须显式

写"既有标注全部保留，只做追加，禁止改写已存在条目的文本"。第三，最终验收等于累积清单全命中加独立审计的遗漏检测维度。

但这只是规则层。真正关键的一步在当天稍晚的时候发生：用户指令"把累积核对做成强制层"——因为"提醒 AI 别忘了"这种事 AI 自己就会忘。于是在同一天，三条配套机制落地：

- 建了一个累积标记清单文件（`cumulative-markers.txt`），记录所有战略基线文件里不可丢失的关键标记。
- 写了一个检查脚本（`check-cumulative-markers.sh`），自动逐条核对标记是否还在对应文件的对应位置。
- 在项目级 hooks 配置中注册为基线守卫——当任何战略基线文件被修改后，自动触发累积核对脚本；发现标记丢失，当场拦截，不允许继续。

这是 DevDesk 四条项目级 hook 中唯一一条有"拦截权"的——其余三条（合并后门禁提醒、提交前先红后绿提醒、改源码前 plan 先行提醒）只提醒不阻塞，唯独这一条是"丢东西就拦"。设计取舍的理由很清楚：提醒太多等于没有提醒，拦截只能花在最贵的事故上——而"文档内容被静默覆盖"的后果是战略基线失真，后续所有基于文档的开发都会走偏，成本是所有事故类型中最高的。

更值得记一笔的是：这个机制上线当天的第一次实弹运行就发现了自己的设计缺陷。hook 注册后，主会话立刻走了一轮"改基线文件、触发守卫、跑核对脚本"的端到端验证。结果发现：如果守卫脚本在会话启动之后才被创建出来，hook 引擎不会在本轮会话中装载它——即"装载时机"问题。CLAUDE.md 中已有的测试纪律要求 hook 变更必须走管道实测（合成测试输入喂给脚本，验证拦截和不拦截两条路径都正确），间接覆盖了"改了 hook 就得验证"的约束，但"重启会话才会装载新 hook"本身目前是经验认识、尚未作为独立规则条目落盘文件。机制上线本身也是一次"发现问题、订正"的微闭环——这个坑记录在工单已知限制里，等待后续正式沉淀。

从事故（四轮融合覆盖、标记丢失）到规则（累积清单逐项回归）到强制层（基线守卫 hook + 检查脚本）到自我订正（装载时机问题被发现），整套流程在 2026 年 10 月 21 日当天全部落盘到位。CLAUDE.md 的 hooks 段落在当日提交（commit `39340e75`）；守卫脚本、检查脚本、累积标记清单等配套文件作为同日落盘的基建已存在于工作目录中，版本提交随整批定账一并走。

这条经验的"保鲜期"目前为零天——因为它刚刚诞生于 2026 年 10 月 21 日。但它已经展示了 DevDesk 经验体系的完整运作逻辑：事故先被提炼为文字规则（写入 CLAUDE.md 和 MEMORY.md），再被翻译成机器可执行的检查（写进 hook 脚本），最后在机制自身的运行中暴露出新问题并被订正。经验从一个"人脑里的教训"变成了一段"开机自动跑的代码"。

6.5 小结：系统记忆

本章从三个层次、四个环节、四个案例，讲清楚了 DevDesk 的开发经验是怎么从"人脑记忆"变成"系统记忆"的。它的核心逻辑可以用一句话概括：经验的价值不在"写下来"，在"每次 AI 干活前都能读到"。

CLAUDE.md 管硬纪律、MEMORY.md 管软纪律、跨项目经验库管通用知识——三层各司其职，但加载时机完全一致：会话启动时全文注入。事故触发识别、提炼成纪律、标注保鲜期、最后靠 `@import` 机制激活——四环走完，一条事故就永久地改变了后续所有 AI 会话的行为边界。

cs-retro 自动复盘作为辅助引擎，在"被动学习"之外补了一条"主动扫描"的通道——让 AI 自己翻自己的对话记录，找那些还没出事但"感觉会出问题"的模式。两者的产出最终都汇入同一个经验池。

经验体系本身也需要经验的修正——累积核对清单案例证明了这一点：一个用来防"经验被覆盖"的机制，在上线第一天就发现了自己的部署时机缺陷。这不是机制设计的失败，恰恰是机制健康运转的信号——它有能力发现自己的问题并自我订正。

截至 2026 年 10 月 21 日，DevDesk 的启动上下文已经包含超过 120 条跨项目通用经验、约 20 条项目专属软纪律、以及一份持续增长的项目硬纪律清单。这组经验的总量已经远超任何一个人的记忆上限，但每一次新会话开始时，AI 都带着完整的"已知坑手册"上线——这正是"机制而非人的记性"这句话在经验管理层面的真实含义。

表 6-1：代表性经验条目"从事故到条目"完整档案

（选 10 条本项目实证扎实的条目，按事故发生时间排列）

#	事故名	发生日期	事故摘要（一句）	提炼的纪律（一句）	写入文件	保鲜标注
1	UAT 真机三连爆	2026-10-19	全量 1600+ 测试全绿当天，用户真机连爆三个 bug（老库缺列、网关拒模型名、字体度量致弹窗叠加），三者全在测试覆盖的缝里	测试按场景写、每条注明防什么事故；交付报场景清单不报数字	CLAUDE.md 验证原则段 + MEMORY.md 子条目 `devdesk-test-by-scenario` / `devdesk-delivery-standard` / `devdesk-all-features-must-run-live`	2026-10-19
2	修复合并不等于交付	2026-10-19	修复已在 dev 分支且推了远端，用户装的旧包照样炸——对用户来说修复不存在	新包打出来放桌面 + 用户真机 UAT 通过才算交付	CLAUDE.md 交付流水线规则 4（阶段 5 注释） + MEMORY.md `devdesk-delivery-standard`	2026-10-19
3	[1M] 网关拒模型名	2026-10-19	智能体 mock 测试全绿，但真网关（公司内部模型网关）拒绝带 `[1M]` 后缀的模型名，草稿生成当场失败	智能体类测试两段制——mock 验证理想路径，收口必须真模型端到端实测	CLAUDE.md 验证原则段 + MEMORY.md `devdesk-llm-test-mock-then-real`	2026-10-21
4	UAT6 追加消息漏做	2026-10-21	主会话给在飞子代理追加的两条任务（md 副题删除、claude 微批后三件）被静默吞掉——代理已完成收尾、消息投递窗口已过	追加即登记、批报告逐项对账、临近完成的批不再追加	跨项目 memory 03-ai-orchestration.md"追加消息会被完成中的子代理吞掉"	2026-10-21
5	多轮融合覆盖前轮注入	2026-10-21	同一文档四轮派单注入，后轮改写条目时挤掉前轮 7 个标记，单轮验证全绿无人察觉	同文件多轮派单后累积清单逐项回	CLAUDE.md hooks 段 + MEMORY.md `multi-wave-same-file-merge-	2026-10-21

				归，后轮 prompt 禁改写既有条目	cumulative-checklist` + hooks 守卫脚本	
6	主会话直接写代码	2026-09-28 / 2026-10-21	主会话既拆需求又改代码，失去第二方视角；后又把 hooks 落地当"配置任务"亲手写了文件	主会话只编排不实施，一切文件落盘改动一律派子任务（含配置和 hook 脚本）	CLAUDE.md 派单与并发段 + MEMORY.md `main-session-no-implementation`（含再犯记录）	2026-10-21
7	手敲 cmake 造三套不一致构建目录	2026-09-28	机器上同时存在 `build/`、`build-release/`、`build-debug/` 三份手工构建目录，内容互相不一致，谁也不知道哪份是"对的"	构建和测试一律走封装脚本，禁止手敲 cmake 命令	CLAUDE.md 常用命令段	2026-09-28
8	12 个残留 worktree	2026-09-28	合并完成后的临时工作目录（worktree）长期未清理，一次清出 12 个（含 EUI-NEO 仓库的残留），占用磁盘且污染 git worktree 列表	worktree 合并后立即删除并 prune，禁止残留	CLAUDE.md 交付流水线规则 4	2026-09-28
9	同文件双写冲突	2026-10-21	两个子代理同时改建 `settings_module.cpp`，漏检交叠白名单，只能强制终止其中一条线	派单前检查在飞白名单做文件级交叠检查，同一文件同一时刻只有一个写者	CLAUDE.md 派单与并发段	2026-10-21
10	标题栏无底色等	2026-09-28	标题栏无底色、详情面板 60% 空白、卡片缺边距——全部是	凡涉及可见 UI 的	CLAUDE.md 功能开发生命周期阶段 4 注释	2026-09-28

UI 问题用户肉眼先发现	用户肉眼先发现，AI 事后才验证	改动，实现者必须自己启动 app 截图看效果再交付

说明：

- "保鲜标注"为条目已知的 `last_verified` 日期。表中所有条目均诞生于 2026 年 9 月底至 10 月 21 日之间，截至制表日均未经历首个 90 天保鲜周期，当前标注值为写入日期——后续审计中以实际核查日期更新。
- 写入位置为截至 2026-10-21 的实际落点。经验条目可能在后续整理中按领域重新归类（如从 MEMORY.md 子条目升格到 CLAUDE.md 硬纪律），此类变动以对应文件的 git 历史为准。
- 跨项目经验库条目（第 4 条）的来源项目为 DevDesk，事故实证在本仓库，仅存储位置因泛用性检查通过而放入全局层。

第7章 AI 行为监控：多层防御的审计体系

一句话使命：讲清楚"一群 AI 在干活，怎么保证不出乱子"——不是靠信任，是靠"不同 AI 互相检查、每层检查有不同视角、数字不达标就拦住不放行"。

7.1 核心设计原则：实现者不等于审查者

DevDesk 的开发模型有一个根本矛盾：代码全由 AI 写，但用户不读代码，无法像传统开发那样靠代码审查来兜底。如果同一个 AI 既实现又审查自己的代码，等于让一个人给自己打分——没有信息增量。因此，DevDesk 的审计体系建立在一个铁律之上：**AI 可以犯错，但错误必须在到达用户之前被至少一道独立于实现者的关卡拦住**。

这一原则在项目中的落地形式很直接：任何改动的实现者和审查者必须是两个不同的 AI 会话。实现者在隔离的工作区里完成代码和测试，然后"交卷"；另一个 AI 只读代码、不做修改，对这份答卷逐条核对。审计者看到的代码状态与实现者完成时一致——它不能"帮着改"，只能"指出来哪有问题"。这种"第二方视角"是整套监控体系的信息来源：审计员不会默认实现已经正确，也不会被实现者的思维路径带着走，它从零开始对照需求、代码、测试和文档，独立形成判断。

这一原则反过来也定义了主 AI 会话的职责：主会话自己不写代码，只做需求裁决、任务拆分、派单、审计结果验收和合并决策。一旦主会话"顺手"直接改文件，就失去了派独立审计员的意义——它自己写的代码，自己再派审计员去查，看似走了流程，实际没有新的视角加入。

7.2 多层审计体系：四层防御塔

DevDesk 的审计不是单次检查，而是一座四层的防御塔。每一层有不同的审查时机、不同的审查者身份、不同的审查范围，它们一起构成了一道"漏过的缺陷往上升一层继续被检查"的梯次防线。图7-1 展示了这座塔的结构。

图 7-1 多层审计防御塔



图注：红色虚线箭头表示上层审计漏过的缺陷继续往上升，直到用户验收层由人眼最终拦截。图注：本图采用功能配色区分四层审计防御（蓝/橙/绿/紫），非装饰用途。

图 7-1 多层审计防御塔

7.2.1 第一层：独立审计关卡（每个改动的必过关）

这是最基础、也是调用频率最高的一层。任何一个子任务完成代码改动并提交后，主会话会立即另派一个只读审计子任务——审计者不参与本轮改造的任何代码实现，是实现者之外的独立视角。

审计者按六个固定维度（正确性、回归面、平台对称性、静默失败面、测试有效性、文档口径）逐条核查。每个维度的完整定义与操作流程详见第4章阶段2。

流程强制层（hooks 守卫）详见第 5 章 5.6——当基线文件被改坏时，机器闸门在审计者到达之前就拦住了。

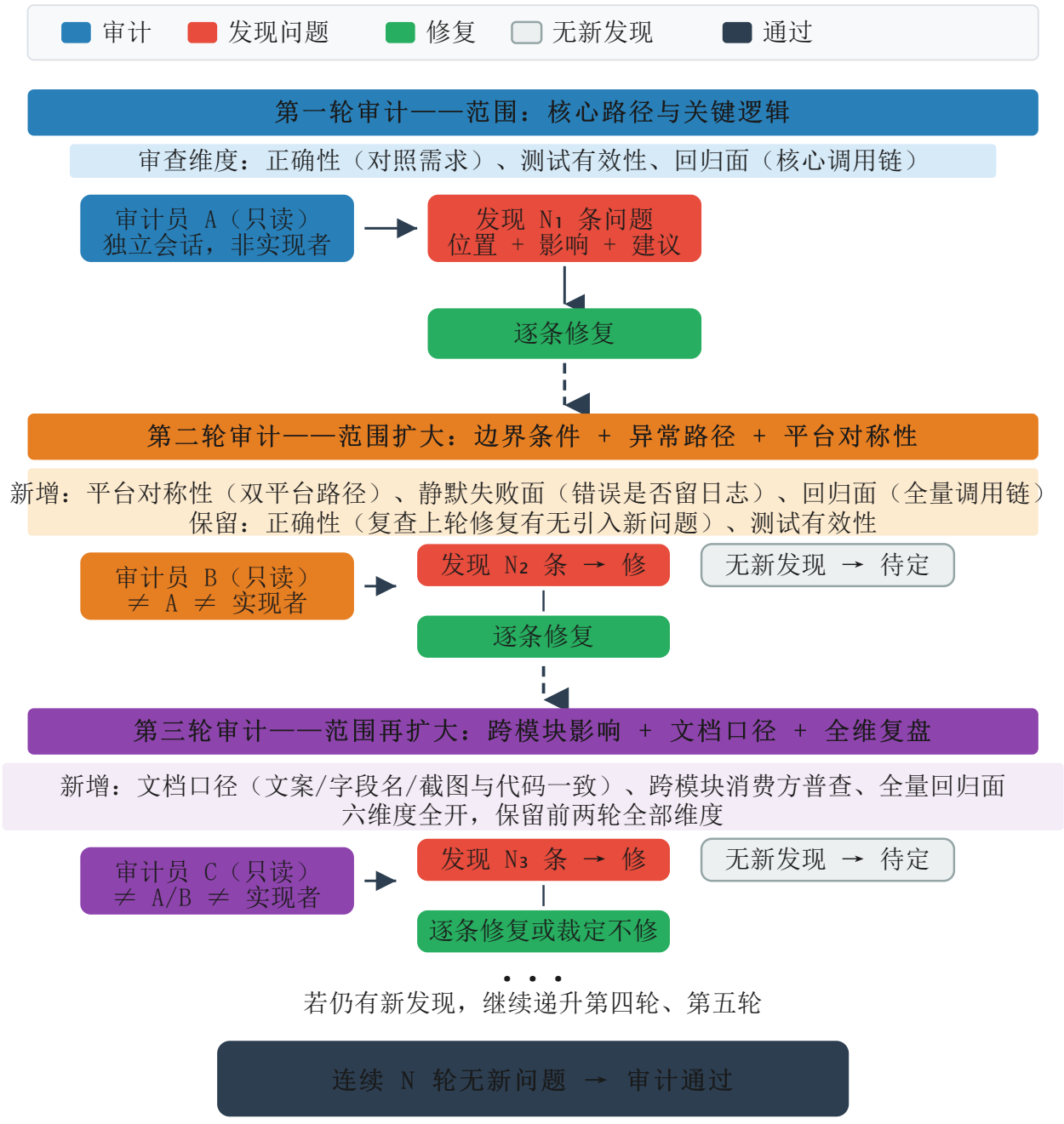
通过判据很明确：六个维度必须逐条给出结论——"未发现问题"也要写明"已核"，每条发现必须带代码文件路径和行号。任一维度未作答、或发现缺少定位信息，审计即标记为不通过。只有六个维度全部核完、所有发现都已落盘，实现者才能进入下一阶段。

7.2.2 第二层：多轮递升 PDCA 审计（里程碑级关卡）

（本节机制经跨项目经验库沉淀、被本项目继承为纪律。）

独立审计管的是单个改动的质量，但一个功能模块通常在多个子任务并行完成后才进入收口阶段。这时候需要启动第二层防御——多轮递升审计。这个名字的含义是：第一轮审计启动时，审查范围聚焦在核心路径和关键逻辑；发现问题后修复，第二轮审计的范围自动扩大——从核心路径扩展到边界条件、异常路径、跨模块影响；修复后再启动第三轮，范围进一步扩大到文档一致性、平台对称性等外围维度。每一轮都是新的审计员（不同于上一轮的审计者），每一轮的范围都比上一轮更大，直到连续若干轮没有发现新问题，审计才算通过。图7-2 展示了这个递升流程。

图 7-2 PDCA 多轮递升审计流程



图注：每轮独立审计员（A/B/C），范围逐轮递升，连续 N 轮无新发现才放行。

图注：本图采用功能配色区分三个审计角色（A蓝/B橙/C紫）及状态节点，非装饰用途。

图 7-2 PDCA 多轮递升审计流程

这种"递升"设计的价值在于：任何单一的审计员都有视野盲区——有人更关注逻辑正确性，有人更关注架构整洁度，有人更关注边界条件。单轮审计的盲区如果不再被复核，就会成为系统的盲区。多轮递升用"不同的人、不同的范围、不同的时间"三次独立观测，把盲区压缩到最小。但它也有成本——每一轮都需要独立的审计会话，多轮下来的时间消耗是单轮的数倍。因此 PDCA

审计只在里程碑收口阶段启用（一个功能模块全部子任务完成后的集中审计），不用于每次小改动。

7.2.3 第三层：追加消息对账（运行时关卡）

这是四层里最轻量但最容易漏掉的一层，来自本项目在 2026 年 10 月 21 日 UAT6 测试中的两起真实事故。

在日常开发中，主会话经常会向已经在执行中的子任务追加新的指令——"刚才漏了一条，你把页面上那个副标题也删掉"。问题是：子任务在收到这条消息之前，可能已经进入了收尾阶段。消息投递机制与子任务的收尾流程存在时间上的竞态——消息在子任务的"下一个工具调用轮"才投递，如果子任务已经完成了最后一个工具调用、正准备向主会话发完成报告，追加的消息就会被静默忽略。子任务报告"完成"，主会话以为追加的指令也执行了——实际没做。

在 UAT6 测试中，两次出现这种情况：一次是追加了删除副标题的指令，另一次是追加了三项后续操作。两次都是子任务报告完成后，主会话核对发现追加项不在报告里，追问子任务才确认"没有收到"。如果不是主会话主动对账，这两个遗漏会一直带到最终交付包中。

解决方案不是去修复消息投递机制——投递时序的竞态是分布式系统的基本约束，修不了。解决方案是"对账"：每追加一条任务，立即在核对清单里登记；子任务的完成报告回来后，先做"报告内容与核对清单"的逐项比对，缺项自动唤起子任务补做，而不重新派新任务。这个机制很小——就一个清单和一条核对步骤——但它补上了"追加后无人确认"的缺口。

7.2.4 第四层：无人值守 watchdog（长时间运行关卡）

（本节机制经跨项目经验库沉淀、被本项目继承为纪律。）

前三层审计都是"事件触发"的——有改动才审、到了里程碑才审、收到报告才对账。但还有一种情况：用户下班了，AI 在无人值守的状态下持续运行，派出去的多个子任务在并行工作。如果某个子任务在执行过程中卡死了（例如系统休眠导致进程挂起、网络超时导致连接僵死），主会话并不知情——它还在等那个子任务的完成通知。

watchdog 是应对这种情况的机制：一个轻量的定时检查脚本，定期（例如每半小时一次）扫描所有在飞子任务的状态——用探针轮询每个子任务"是否还活着"，判断在指定时间内有没有产出新进展。对于"活着但停滞"的任务（还有心跳但长时间没有工具调用），先尝试发消息唤醒续跑；对于"已失去心跳"的任务，判定为卡死后终止，并从同一断点重新派发新任务接替。

这个机制的关键是"不在源头上堵死错误，而是持续检查并自动修复"。不靠"一次派发就保证永远正确"——因为长时间运行的不可控因素太多了。靠的是"持续检查、发现问题、自动修复"。

7.3 审计产物的结构标准化

四层审计的产出格式各不相同，但它们共享一个底层要求：每一条发现必须能独立追溯、独立处理、不依赖审计者的上下文才能理解。这个要求在"独立审计关卡"一层落地为强制结构：

1. **精确位置**：代码文件路径加行号。没有行号的发现等于让实现者自己去代码里找问题在哪——不同人定位同一个问题可能找到不同位置，追踪和复核都会断链。

2. **问题描述**：用一两句话说清楚"代码当前在做什么，应该做什么"，而不是写"这里有个 bug"。描述必须让一个完全不认识这段代码的人（包括实现者在几天后回来复查时）能看懂问题是什么。

3. **影响面**：这个问题不改会导致什么——是功能缺了某一块、还是某条路径静默失败、还是两平台行为不一致。

4. **修复建议**：给出最直接的修改方向（不是具体代码，而是"应该往哪个方向改"）。

5. **六个维度的逐条结论**：每个维度的检查结果必须明确写入——"已核"不等于"没问题"，"没问题"也不等于"已核"。宁可写明"正确性维度：已核，未发现问题"，也不能让维度空着。

这种标准化不是形式主义。在 DevDesk 的实际开发中，同一批改动经常经历多次审计——独立审计、PDCA 多轮，以及从兄弟项目继承的 scope-drift 专项审计——不同审计员的产出如果格式不统一，就无法做横向比对（"上一轮发现的那条，这一轮修了没有？"）、也无法做纵向追踪（"这个发现是哪个阶段查出来的？"）。标准化格式让审计产物从一个审计者的"一过性笔记"变成一份可以跨轮次、跨审计者、跨时间追踪的"处置流水账"。

7.4 边界与局限

这套四层审计体系能防住什么？能防住"AI 犯了人类能想到检查的那些错误"——逻辑漏了某条需求、改了 A 忘了改依赖它的 B、测试其实没测到关键路径、文档和代码对不上。这些都是"已知可查"型错误：人类在设计审计维度时心里已经有了"可能在哪出错"的假设，审计就是去验证这些假设。

它防不住什么？防不住"人类也没想到要去检查的那类错误"。真机 UAT 三连爆就是典型案例：在 1600 多条测试（2026-10-19 当日）全绿的当天，用户在真机上连爆三个问题——老数据库缺少新建的列导致任务卡片创建失败、远程网关拒绝某个特殊格式的模型名称导致草稿功能崩溃、Windows 的字体度量算法与 macOS 不同导致弹窗文字叠加显示。这三个问题没有一个在预先设计的审计维度中——设计审计时，没有人在清单上写"检查新建列在老数据库上是否存在""检查模型名称格式是否被网关接受""检查字体度量在两平台上是否产生不同视觉效果"。

这不是审计体系的设计缺陷——审计体系的上限就是"设计它的人能想到的范围"。超出这个范围的错误，唯一的防线是第 3 章讲的用户真机验收：人的眼睛和手感是最终安全网。审计体系

和用户验收之间的关系不是"前者替代后者"，而是"审计把已知可查的错误挡在用户看到之前，让用户的注意力和时间花在那些只能由人眼发现的问题上"。

7.5 纪实案例

以下两个案例展示审计在实际运行中是如何拦截问题的——一个来自本项目，一个来自跨项目经验库的沉淀。

案例一：智能体详情弹窗审计发现待确认边界与回归冲突（本项目，2026-10-19）

背景：智能体模块的详情弹窗正在进行布局改造，需求文档列出了六条具体的改动项。实现者完成任务并提交代码后，主会话派出了独立审计员。

审计员采用逐条核对的方法：把需求文档的六条内容逐一与代码改动对照。核对到第三条"配置条改 chip 可折行"和第四条"停止 chip 文案缩短"时，审计员发现代码中未体现这两条改动——但经核查后认定，这两条按任务分界（P1 布局壳与 P3 配置浮层）属后续批次的范畴，不构成本批次缺口。审计员在报告中主动标注了归属待确认状态并提请主会话澄清：移交文档在批次间未标注分界，导致"该不该做"本身成为待决问题。这不是实现者偷工减料——而是需求边界在批次交接时未标注清楚。审计的价值正在于在代码进入下一阶段之前发现这类"边界未清"的问题。

同时，审计员在测试层发现：因为布局尺寸从旧的 282 像素改成了 300 像素，原有测试文件中的九条断言全部指向旧尺寸值——它们会全部报错，但实现者没有同步更新。这九条回归面冲突如果不修，测试将无法反映新代码的真实状态。审计报告逐条列出了精确位置和应该改成的新值，实现者按清单修完后才被允许进入后续阶段。

案例二：范围外变更审计拦截"顺手"删除（跨项目经验库沉淀）

（以下案例来自跨项目经验库，被本项目继承为纪律。）

在一次后端接口的功能改造中，一个子任务在提交代码时，除了实现需求内的功能改动，还"顺手"删掉了一段看起来无关的代码。实现者判断这段代码"不再需要了"——但这段代码是另一个模块的依赖入口，删除后那个模块的页面上某项功能直接消失，没有报错、没有任何日志。

在这次事故之后，审计体系增加了一条"只审范围边界"的专项维度——不检查代码修改得对不对，只检查"改了什么不该动的东西"。每次审计不仅要看实现是否完整，还要用改动前后的文件差异，逐行核对：哪些改动是需求明确要求的、哪些是"顺手"加的、哪些是"顺手"删的。所有不在需求范围内的改动，无论看起来多合理，一律标记为"范围外"并要求实现者说明理由——不能说"我觉得应该删"，必须给出"不删会怎样"的技术推演。这项纪律被写入了跨项目经验库，随后被本项目全额继承。

表 7-1 审计六维度检查清单

维度	检查项	对应反例事故关键词
正确性	对照用户原始需求逐条核对代码行为，不依赖实现者自述——含需求文档描述与代码行为比对	需求六条只做了四条，实现者报告写"完成"但逐条核对发现漏项
	需求中明确标注"暂不支持/不在范围"的功能点，实现是否越界做了	审计发现实现顺手改了需求范围外的功能，未获授权
	分支逻辑的"成功路径"和"失败路径"都覆盖了需求	只做了正常路径，异常路径被忽略
	实现者回单中的"做了什么"说明与代码 diff 吻合	说明写改了三处，实际 diff 看只改了两处
回归面	改动触发的所有调用点/依赖点已同步更新	搬容器漏搬锚点引用，页面 60% 空白
	改动了共享数据结构/常量/配置，所有消费方已适配	常数 282 改成 300，9 条测试断言仍断言旧值
	改动未引入对既有用户数据的兼容破坏	新增字段后旧数据库打开报错（UAT 三连爆其一）
	改动无关的代码路径未被触及	插一个函数时顺带删掉两段无关代码
平台对称性	改动在 Windows 和 macOS 两平台的编译路径下都存在	Windows 开发期写的 macOS 代码从未编译，回 macOS 后 100+ 用例失败
	两平台的平台特定代码（`_win.cpp` / `_macos.cpp`）都更新	只改了 Windows 实现文件，macOS 实现文件没改
	两平台的用户界面表现一致	字体度量算法不同导致弹窗文字叠加（UAT 三连爆其三）
静默失败面	错误路径有日志输出，不是吞掉异常后静默返回	异常被空捕获块吞掉，一行日志都没有
	失败情况有界面可见的提示信息	功能"做了但没生效"，用户完全不知道
	兜底值和截断行为有合理理由且记录日志	高度不足被截断，少了几行不报错
	数据库迁移/升级的失败路径有明确错误信息	旧库缺列导致建任务失败（UAT 三连爆其二）
测试有效	新增断言在改动回退时确实会变红（非恒真）	自定义检查只打印失败信息不抛出异常，报告"7/7 全绿"

性		
	测试覆盖了实际业务流程（真窗口、真网络），不仅是纯逻辑	1600+ 条全绿当天真机三连爆
	涉及界面的改动有真窗口探针覆盖	断言"看起来对"而不是"数值等于期望值"
	每次改动对应的测试用例标注"防什么具体事故"	测试注释写"覆盖 XX 函数"而不是"防 XX 场景"
文档口径	文档中的字段名、状态值、行为描述与代码一致	文档写了不存在的字段名
	文档中的版本号、日期、数字与实际一致	文档写"0 未提交改动"而实际有 3 个未跟踪文件
	审计报告中引用的代码行号真实存在	引用的行号在目标文件中不存在对应代码
	图表/截图/示例中的数据与当前实际运行结果一致	截图为旧版本，新版界面布局已不同

第8章 成果与可复制清单：别人怎么照抄

DevDesk 一个月密集迭代后，摆在面前的是六个业务模块全功能运行、双平台原生支持、CI 自动构建与测试、安装包交付的完整产品。约 1600 余条自动化测试（截至 2026-10-19）、共计约 1910+ 次提交（截至 2026-10-21）、110 余条可用功能（截至 2026-10-21 清单口径）——这些数字本身不是成果，它们只是"机制有效运转"的可观测痕迹。

真正的成果是一套开发体系：六阶段流水线（第 4 章）、四律交付纪律、20 条预防机制（第 5 章）、九大类跨项目经验库（第 6 章）、四层审计防御塔（第 7 章）——以及它们的共同锚点，项目级硬纪律文件 CLAUDE.md 与 2026-10-21 落地的 hooks 强制层。这套体系就是本章要回答的问题：别人拿走什么、按什么顺序搭建，才能在本项目上复制出同样的效果。

8.1 方法论本质：不是"AI 写代码"，是"机制约束 AI 写代码"

如果把 DevDesk 的开发方法理解成"让 AI 写 C++"，那等于只看到了水面上的十分之一。水面下的十分之九是：用户定义需求边界，AI 在机制约束下自主执行，多层 AI 互审代替人审，经验从事故中自动沉淀到启动上下文。四个环节缺一不可。

打个比方：用户相当于给建筑定了"几层楼、什么用途、什么风格"，然后一套自动化施工流水线接手——挖地基的不是同一批机器、砌墙的不是同一批机器、质检的不是同一批机器、最后交房给用户验收。用户不需要知道混凝土标号，但他必须知道"先质检再封顶"这条工序不能跳——因为历史上有过没质检就封顶、后来发现墙体开裂只能拆了重来的事故。

这就是为什么本报告的可复制性**不取决于用户有没有技术背景**，而取决于有没有把这套机制完整复制过去。拿走一条机制而漏掉另一条，系统会退化——比如拿走 worktree 隔离但漏掉独立审计，"AI 在隔离环境里写代码"就降级成"AI 在隔离环境里写可能有问题的代码但没人检查"。可复制性的关键不在每条机制单独的效果，在于它们之间的依赖关系——缺一个，整个链条的保证强度就塌掉那一段。

8.2 可复制清单

以下清单按"缺一则系统退化"的强度分三层：必备机制（6 条，缺一则系统退化为"AI 随手改代码"的裸奔状态）、推荐机制（5 条，有更好没有也能跑）、配套基建（3 条，让系统能实际运转的物理条件）。编号顺延，每条给一句话说明和分层理由。

必备机制（缺一则系统退化）

1. 项目级 CLAUDE.md——AI 启动时自动加载的硬纪律文件。内容包括行为边界（什么能做、什么不能做）、反事故红线（哪些操作绝对禁止）、已知坑清单（过去每一条事故的教

训）。分层理由：这是整座机制大厦的地基——没有它，AI 不知道任何规则，凭直觉写代码，等于裸奔。这也是起步指南的第一步（见 8.3 节）。

2. **Worktree 隔离**——每个代码改动在独立的代码副本中进行，互不污染。分层理由：第 5 章 5.2 的实证——三个子任务在同一个工作树里同时写码，结果 `git status` 一塌糊涂，三条线的改动搅在一起分不开。不隔离就谈不上并行开发，而 AI 驱动开发的效率核心就是并行。

3. **独立审计关卡**——每个改动完成后，由一个不同于实现者的 AI 会话做六维度只读审查（正确性、回归面、平台对称性、静默失败面、测试有效性、文档口径）。分层理由：第 4 章阶段 2 有完整的六维度机制展开，第 7 章 7.2.1 在多层防御塔结构中定位了独立审计的角色——这是"实现者不等于审查者"铁律的落地形态。没有独立审计，错误在到达用户之前没有第二双眼睛拦过。

4. **先红后绿**——修复类改动必须证明测试"会失败"再修到通过。分层理由：第 5 章条第六条和 Ch4 阶段 4 的测试有效性维度都用实证说明了恒真断言的问题——自定义 check 只打印不抛出异常，报告"7/7 ALL PASSED"但全部失败。没有先红后绿的要求，假绿测试会在系统中累积，直到真实事故暴露。

5. **经验沉淀闭环**——每次事故提炼为一条纪律，写入 AI 每次启动时自动加载的文件，后续所有会话自动继承。分层理由：第 6 章的全章论述——经验的价值不在"写下来了"，在"每次 AI 干活前都能读到"。没有闭环，事故教训只存在发生当天的那条会话里，下次启动已归零。

6. **流程强制层（hooks + 守卫脚本）**——把"要求 AI 走流程"翻译成机器闸门：基线文件内容被改坏当场拦截、合并后必过门禁、提交前递先红后绿提醒。分层理由：第 5 章 5.6 整节展开——规则写在文件里等 AI 读，和规则由脚本自动拦截，是两种完全不同的保证强度。第 6 章案例四（累积核对清单强制化）证明了把"多轮融合覆盖"事故翻译成机器检查脚本后，人力不可靠的验收从此由机器接管。最小形态 = 一个累积核对脚本 + 几条项目级 hook，半天级搭建。

推荐机制（有更好，没有也能跑）

7. **跨项目经验库**——按领域分文件（交付纪律、反幻觉、AI 编排、用户协作等），通过 `@import` 指令在 AI 启动时全量注入。分层理由：第 6 章 6.1 节经验三层体系中的底层层——有它就能跨项目共享教训（DevDesk 继承时已有约九十条条目），没有它也仍有项目级 CLAUDE.md 打底。

8. **自动复盘（cs-retro 类）**——定期扫描对话记录，自动识别重复出现的问题模式和被用户纠正的错误类型，提炼候选经验条目。分层理由：第 6 章 6.3——手动沉淀靠"事故发生了有人想起来写"，自动扫描补了一条"还没出事但感觉会出问题"的主动通道。有它能让经验积累加速，没有它纯靠人的记性去触发沉淀，速度慢但核心闭环仍在。

9. **多轮递升 PDCA 审计**——里程碑级收口关卡。每轮审计范围自动递升（第一轮核心路径，第二轮边界条件，第三轮跨模块影响），直到连续若干轮无新问题才通过。分层理由：第 7 章 7.2.2——单轮审计有盲区，多轮不同审计员的不同视角把盲区压到最小。但成本是单轮的数倍，只适合里程碑关口，不适合每次小改动。

10. **无人值守 watchdog**——长时间运行的心跳检测。定期扫描所有在飞子任务，心跳正常的放行，停滞的尝试唤醒续跑，卡死的终止并重新派发。分层理由：第 7 章 7.2.4——不靠"一次派发永远正确"，靠"持续检查、自动修复"。只有在"用户下班、AI 还在跑"的场景下才有实际需求。

11. **交付前三问**——发任何安装包前，必须回答三个问题：存量数据升级路径验过吗？真实外网 / 网关验过吗？目标平台真机验过吗？答不上来写"未验证"。分层理由：第 5 章条目九和第 4 章阶段 4——这是 2026-10-19 真机 UAT 三连爆的直接产物。没有交付前三问，会在"测试全绿"的幻觉中把打不开的包送到用户手里。

配套基建

12. **编译缓存（ccache）**——多工作区并行编译的前提。让不同 worktree 的同一份源码走同一份缓存，首次构建后的增量构建从十分钟级降到分钟级。分层理由：第 2 章 2.2——没有缓存时，每个新 worktree 都是全量冷编译，并行开发的效率会被编译时间拖垮。

13. **构建脚本封装**——禁止手敲构建命令，一切走封装脚本。脚本是唯一被反复执行的构建路径——CI 用它，worktree 用它，主会话用它。分层理由：第 5 章条目十一的实证——手敲命令造出三份互相不一致的构建目录，排查浪费了一个多小时。"差不多"就是不一致，不一致就会在某个时刻以莫名其妙的结果出现。

14. **真窗口探针（probe）**——模拟真实鼠标点击、键盘输入、拖拽事件和界面渲染的自动化测试。启动真实窗口（含 OpenGL 渲染后端），注入事件，读取像素坐标与几何数值做断言。分层理由：第 4 章阶段 4——单测能验证"代码自洽"，验证不了"按钮画在正确坐标上"。这是 GUI 类项目独有的需求，非 GUI 项目可跳过。

表 8-1：可复制清单完整版

分类	编号	机制名	一句话说明	为什么这层	搭建要点	对应章引用
必备	1	项目级 CLAUDE.md	AI 启动时自动加载的硬纪律文件，内含行为边界、反事故红线、已知坑清单	地基——AI 不知道任何规则即裸奔	项目概况+工具链+核心约束三项即可起步，后续随事故增长（小时级）	第5章 5.6 注入层；第6章 6.1 第一层
必备	2	Worktree 隔离	每个改动在独立代码副本中进行，互不污染	不隔离谈不了并行——三条线同目录写码搅成一团无法提交	Git worktree add + 脚本管理；用完即清 规则同步落地（半天级）	第5章 5.2 条目三；第4章 4.3 调度纪律
必备	3	独立审计关卡	实现者≠审计者，六维度只读审查	失去第二方视角，错误直达用户——既写又审没有信息增量	定六维度清单+findings 文件格式；第一个审计员 prompt 模板最重要（半天级）	第4章 阶段2；第7章 7.2.1
必备	4	先红后绿	修复前必须证明测试会失败再修到通过	恒真断言=假绿——自定义 check 只 print 不 raise，失败计数永远 0	修复类改动的提交前验证步骤；hook 提醒辅助（小时级）	第5章 5.3 条目六；第4章 表4-1
必备	5	经验沉淀闭环	事故→提炼纪律→写入启动上下文→后续 AI 自动继承	经验在人脑=人走即丢；只在文档=没人翻即废纸	三条纪律起步（选最痛的三次事故），文件放 AI 启动时自动加载的位置（小时级）	第6章 6.2 四环闭环；第6章 表6-1 档案
必备	6	流程强制层（hooks）	把"要求 AI 走流程"翻译成机器闸门——基线改坏拦截、合并后门禁、提交前提醒	规则在文档里等 AI 读，与规则由脚本拦，是两种不同的保证强度	最小形态=一个累积核对脚本+几条 hook（半天级）；先做唯一会拦的那条，其余提醒逐步补	第5章 5.6 四层机制；第6章 案例四
推荐	7	跨项目经验库	按领域分文件的共享经验，@import 注入	有它=跨项目继承教训；没有也仍有 CLAUDE.md 打底	选 3-4 个领域分文件，从已有项目 CLAUDE.md 拆出通用条目（半天级）	第6章 6.1 第三层；第1章 1.4（经验库起源）

推荐	8	自动复盘（cs-retro）	定期扫描对话记录，自动提炼经验候选	补充"没出事但感觉会出事"的主动扫描通道	先手动沉淀跑通闭环再引入自动扫描（半天级）	第6章 6.3
推荐	9	多轮递升 PDCA 审计	里程碑级多轮审计，每轮范围递升	单轮有盲区——多轮不同视角压到最小	里程碑关口启用，不是每次小改动（小时级配 prompt）	第7章 7.2.2
推荐	10	无人值守 watchdog	定时心跳检测在飞子任务，卡死的唤醒或重派	不靠一次派发永远正确，靠持续检查+自动修复	动态注册表+定时扫描（半天级）	第7章 7.2.4
推荐	11	交付前三问	发包前必答：存量升级？真网关？真机？	假绿测试会把打不开的包送到用户手里——三连爆实证	三个固定问题写进交付检查单（分钟级）	第5章 条目九；第4章 阶段4
基建	12	编译缓存（ccache）	多 worktree 并行编译的前提，全量冷编译→分钟级增量	没有缓存=每个新 worktree 十分钟级全量冷编译——并行退化为串行	配置缓存目录+BASEDIR（让不同 worktree 同源码命中），半天级	第2章 2.2；第5章 5.2 条目五
基建	13	构建脚本封装	单命令构建+测试，禁止手敲 cmake	手敲造出多份不一致构建目录——"差不多"就是不安全	一个脚本收口全部构建参数（小时级）	第5章 5.5 条目十一
基建	14	真窗口探针（probe）	真实窗口+注入事件+像素断言	GUI 项目独有——无头单测验证不了"按钮画对坐标"	探针框架+几个典型场景探针起步（半天级）；非 GUI 项目可跳过	第4章 阶段4 第二层

8.3 起步指南：从零到最小可跑循环

一个新项目从零开始搭建这套方法，不需要一次到位。本节给出四步推进路径——每一步都是"够用即可"的最小形态，后续机制是在被事故撞出来之后才追加的。

第一步：写第一版 CLAUDE.md。 不需要长篇——三条核心信息足够：(1) 项目是干什么的、用的什么技术栈；(2) 构建和测试命令写死（一条命令能完成构建+测试，让 AI 不需要猜参数）；(3) 一两条最核心的约束（"不要改配置文件""模块之间不能互相依赖"之类）。写完放在仓库根目录下，AI 启动时会自动读取。

第二步：建立构建脚本。 把第一步里写的那条构建命令封装成一个脚本文件。不是"建议用脚本"，是"让脚本成为唯一能被执行的构建路径"——第 5 章的手敲 cmake 事故证明，"差不多

的手敲命令"在不同时间、不同人（包括不同 AI 会话）手上会造出不一致的结果。一条脚本管全部参数，从此构建永远一致。

第三步：启用 worktree 隔离。 这是"并行"的前提。第一次派子任务改代码之前，先确保子任务在自己的 worktree 里工作。这一步的配套规则不能漏：worktree 用完立即删除（第 5 章 5.2 条目四——一次清出 12 个残留 worktree 的教训）。

三条到位后，最小可跑循环就转起来了：**用户提需求 → 主会话派子任务（在 worktree 里改码+提交）→ 独立审计（另派一个 AI 只读检查六维度）→ 串行合并进集成分支 → 交付安装包 → 用户真机验收**。这六步对应的正是第 4 章的六阶段流水线——它是被事故撞出来的完整形态，但它的骨架只有上面这三条基建。

第四步：随事故自然生长。 不需要预先把全套 14 项机制设计好。大部分机制都是在"又出事了"之后才追加的——换句话说，机制是被事故"撞"出来的，不是预先设计的。下面的触发清单帮助判断什么事故该补什么机制：

- 第一次出现"AI 顺手改了不该改的东西"→ 补镜像复述纪律（必备 1 的 CLAUDE.md 扩充）。
- 第一次出现"审计测完全绿但用户真机一上手就炸"→ 补交付前三问（推荐 11）和"测试绿不等于可用"的交付口径调整。
- 第一次出现"多个子任务改同一个文件导致合并冲突废掉一整条线"→ 补同文件互斥检查（必备 1 的 CLAUDE.md 扩充）。
- 第一次出现"加了 hook 或规则但 AI 不遵守"→ 补流程强制层 hooks（必备 6）。
- 第一次出现"隔了一周的会话完全不记得上次的教训"→ 补经验沉淀闭环（必备 5）。
- 第一次出现"长时间无人值守的子任务悄悄卡死"→ 补 watchdog（推荐 10）。

图 8-1 把这条"从零到完整体系"的依赖路径和触发条件画成了路线图。图的读法：从左到右按机制层递进（路径一→路径二→路径三），每个箭头表示"先有 A 才能有 B"的依赖关系；最右侧的标注说明"什么时候该拐去补这个机制"。

8.4 适用边界

这套方法不是万能钥匙。它最适合的场景有三条特征同时满足。

其一，用户懂"要做什么"但不需要懂"代码怎么做"的中大型项目。如果用户自己也写代码、读代码，那很多机制（尤其是独立审计）可以由人直接替代 AI 来做，不需要额外的 AI 会话参与。但人审代码不等于本方法失效——本方法的审计六维度、经验闭环和 hooks 强制层对纯人工项目同样有价值，因为它们解决的是"审核者漏看"和"经验断档"问题，而这两个问题在纯人工开发中同样存在。

其二，技术栈有成熟的 AI 训练数据覆盖（C++、Python、TypeScript、Rust 等主流语言）。DevDesk 选择 C++20，是因为这门语言在 AI 训练数据中的密度足够高——AI 生成的代码质量有保障。如果项目用的是冷门语言或自研框架，AI 的代码生成质量本身就不可靠，整套"AI 在机制约束下自主执行"的前提就不成立了。

其三，涉及图形界面或交互的项目需要真机验收环节。这是本方法第 3 章中用户角色的核心——人的眼睛和手感是最终安全网。第 7 章 7.4 节明确指出："审计体系的上限就是设计它的人能想到的范围"，超出这个范围的错误，唯一防线是用户真机验收。

不适用的场景同样明确。极小型项目（单文件脚本、几十行的工具）不需要 worktree 隔离和多层审计——它们的开发节奏本身不会触发这些机制要防范的问题。对实时性要求极高的底层系统开发（操作系统内核、驱动程序）——这类项目要求的是确定性，而 AI 生成的代码天然带有不确定性，本质冲突。

还有一个重要边界：**这套方法的可复制性不要求用户有完全相同的项目形态**。第 1 章的"用户不熟 C++"是 DevDesk 的具体背景，但机制的复制对象不是"不懂 C++ 的人"——是任何"想让 AI 在规则约束下自主执行、不想靠自己的注意力当防错网"的项目。技术栈可以换、项目方向可以换、AI 工具可以换（Claude Code 换成别的同类工具），只要"定义边界 → 机制约束 → 独立审查 → 经验闭环"这条骨架在，方法的实质就在。

8.5 小结

前面七章讲的是"我们怎么做的"。本章回答的是"你们怎么做"。

答案不是"拿走 DevDesk 的代码"，是"拿走这套机制清单，按依赖顺序搭建，在事故的推动下自然生长"。它和传统"最佳实践清单"的区别在于：传统清单的条目是"你应该这么做"，本清单的每条条目都站着至少一个真实事故——它不是推演出来的，是撞出来的。正因为每条机制都有实证，才值得照搬；正因为每处都标注了事故来源，才知道搬错了可以退回去重来。

从一份 CLAUDE.md 起步，到一个可以并行派发 8 个子任务、四层审计防御、经验自动闭环的开发系统——DevDesk 用了一个月。起步只需要一天。

图 8-1：从零起步最小路线图

按依赖关系递进，标注事故触发条件

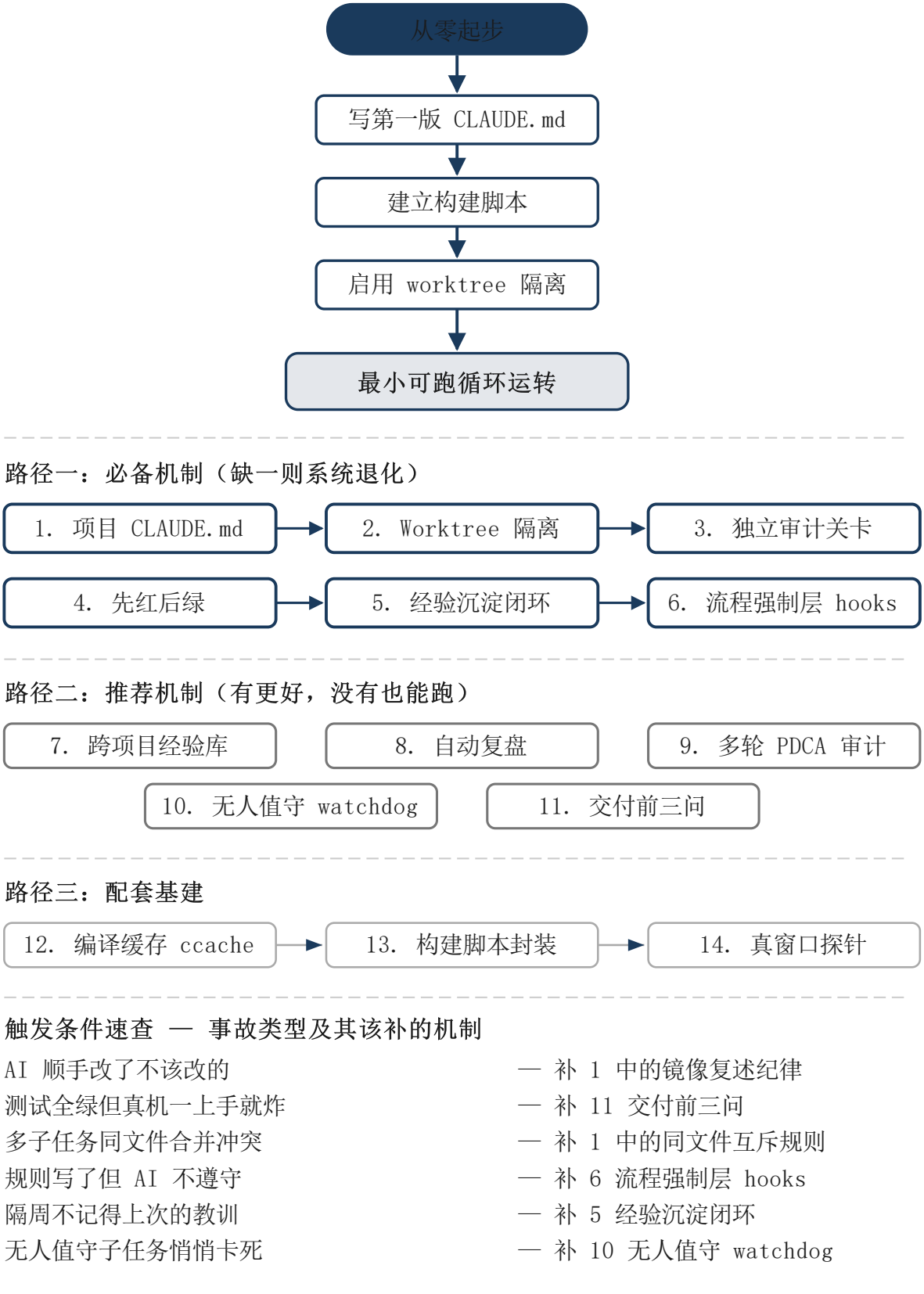


图 8-1　从零起步最小路线图